

TDM Insight

A Native Android Vancomycin Therapeutic Drug Monitoring Calculator

Design, Implementation and Testing Report

Course code	CDE2313
Course title	Mobile Application Development
Programme	Bachelor in Data Science
Academic session	2025/2026
Semester	3
Lecturer	Ts. Mohd Zulkifli Mohd Zaki (Lead); Madam Siti Shafrah Shahawai (Co-Lead)
Assessment	Assessment 2 — Group Project (25%)
Case study	TDM Insight — Native Android Therapeutic Drug Monitoring Calculator
Citation style	APA 7th edition
Date prepared	3 September 2026

Prepared by

No.	Student name	Student ID
1	Mowlid Haibe	
2	Abdinaazir Mustafe	
3	Elham Ahmedngus	

Student ID numbers are to be completed by each member before submission.

Academic prototype. Not a clinically validated prescribing, diagnostic or treatment-decision system. All demonstration cases are fictional.

Table of Contents

1. Introduction.....	6
1.1 Background.....	6
1.2 Purpose.....	6
1.3 Objectives.....	6
1.4 Target Users.....	7
1.5 Scope.....	7
1.6 Structure of This Report.....	7
2. Problem Statement.....	8
2.1 The Clinical Problem.....	8
2.2 Limitations of Current Practice.....	8
2.3 Who Is Affected.....	8
2.4 How TDM Insight Addresses the Problem.....	9
3. Proposed Solution.....	9
3.1 Solution Overview.....	9
3.2 Key Features.....	9
3.3 Scope Boundaries and Deliberate Extensions.....	10
4. System Overview and Requirements.....	10
4.1 Requirements Traceability.....	10
4.2 Functional Requirements.....	11
4.3 Non-Functional Requirements.....	13
5. Technology Stack.....	13
6. System Architecture.....	16
6.1 Architectural Pattern and Justification.....	16
6.2 The Architecture in Detail.....	17
6.3 Layer Interaction.....	18
7. Use Case Model.....	18
7.1 Actors.....	18
7.2 Use Case Summary.....	18
7.3 Detailed Use Case Specification.....	20
8. Component and Package Structure.....	21
9. Behavioural Models.....	21
9.1 Activity Diagram — Application Entry and Authentication.....	21
9.2 Activity Diagram — The Five-Step Calculation Workflow.....	22
9.3 Sequence Diagram — Running and Persisting a Calculation.....	23
10. Data and Database Architecture.....	24
10.1 Entity Relationship Model.....	24

10.2 Table Specifications.....	25
10.3 Access Control.....	26
11. Frontend, Backend and Data Flow.....	27
12. Navigation and Screen Map.....	28
13. Domain Class Model.....	29
14. Input Validation.....	29
15. Authentication, Session Management and Security.....	31
15.1 Registration and Sign-In.....	31
15.2 Session Persistence.....	32
15.3 Credential Management.....	32
16. The Calculation Engine.....	33
16.1 Named Constants.....	33
16.2 Shared Relationships.....	33
16.3 The Pre-Dose Workflow.....	33
16.4 The Post-Dose Workflow.....	34
16.5 The Pre + Post Workflow.....	34
16.6 Worked Examples.....	34
16.7 Explainability.....	35
17. User Interface.....	35
17.1 Design System.....	35
17.2 The Landing Page.....	36
17.3 Application Screens.....	40
17.4 Interaction Details.....	42
17.5 Non-Functional Interface Elements.....	42
18. Testing and Debugging.....	43
18.1 Strategy.....	43
18.2 Test Design Rationale.....	44
18.3 Manual Testing.....	44
18.4 A Known Defect in the Post-Dose Workflow.....	45
18.5 Debugging Practice.....	46
19. Repository Management and Version Control.....	46
20. Responsible Use of Artificial Intelligence.....	47
21. Limitations and Future Work.....	48
21.1 Limitations.....	48
21.2 Future Work.....	48
22. Conclusion.....	49
References.....	50

1. Introduction

1.1 Background

Therapeutic drug monitoring (TDM) is the clinical practice of measuring drug concentrations in a patient's blood and adjusting the dosing regimen so that exposure stays inside a therapeutic window. It is reserved for medicines whose effective dose sits uncomfortably close to their toxic dose. Vancomycin, a glycopeptide antibiotic used as first-line therapy against methicillin-resistant *Staphylococcus aureus* (MRSA), is the archetypal example: too little and the infection is not cleared, too much and the patient risks nephrotoxicity.

The 2020 revised consensus guideline published by ASHP, IDSA, PIDS and SIDP moved vancomycin monitoring away from trough concentration alone and towards the area under the concentration–time curve over 24 hours (AUC_{24}), recommending a target of 400–600 mg·h/L for serious MRSA infection (Rybak et al., 2020). That change is clinically better founded, but it is computationally heavier: AUC_{24} cannot be read off a single laboratory value. It must be derived from a pharmacokinetic model fitted to the patient's demographics, the prescribed regimen and one or two measured concentrations.

TDM Insight is a native Android application that performs this derivation at the bedside. It was developed for Assessment 2 of CDE2313 Mobile Application Development at Albukhary International University, working from the case study issued by the course lecturer (Mohd Zaki, 2026). The application is deliberately narrow in scope: one drug, three sampling workflows, and a calculation engine that is separated from the user interface, thoroughly validated and fully explainable.

1.2 Purpose

The purpose of this report is to document the analysis, design, implementation and testing of TDM Insight as it actually exists in the project repository. Every architectural claim, diagram, formula, database column and screenshot presented here was derived from the source code rather than from a design intention. Where the implementation departs from the original specification, or where a defect is known, the report says so explicitly.

1.3 Objectives

The project set out to meet the following objectives.

1. **Deliver a working native Android application** built in Kotlin with Android Studio, using Jetpack Compose and Material 3, that a pharmacist can operate end to end without instruction.
2. **Implement all three vancomycin sampling workflows** — Pre-dose, Post-dose and Pre + Post — each driven by a distinct pharmacokinetic pathway.

3. **Render dynamic input forms** that expose only the fields the selected workflow requires, rather than presenting every possible field on one screen.
4. **Validate input beyond emptiness checks**, distinguishing blocking errors from advisory warnings and enforcing cross-field timing relationships and mathematical safety guards.
5. **Separate the calculation engine from the user interface** so that the pharmacokinetic logic is pure Kotlin, contains no Android framework dependency and can be unit tested in isolation.
6. **Return intermediate as well as final results**, so the user sees K_e , half-life, volume of distribution and clearance rather than a single opaque number.
7. **Make every result explainable** through a dedicated view that walks from input values through intermediate values and pharmacokinetic parameters to the final AUC_{24} and dose recommendation.
8. **Apply a consistent Material 3 interface** with genuine light and dark colour schemes and a semantic colour convention shared by every validation message.

1.4 Target Users

TDM Insight is written for two overlapping groups. The primary users are **clinical pharmacists and pharmacy students** who need to convert a laboratory report into an AUC_{24} estimate and a dose recommendation quickly, and who need to be able to show their working. The secondary user is the **prospective user or evaluator** who reaches the project through its public landing page and wants to understand what the application does before installing it. Because the application is an academic prototype, no user of any kind may enter real patient data; this restriction is surfaced on first launch, on every input screen and in the About screen.

1.5 Scope

The application models vancomycin only. Gentamicin and amikacin appear on the medication selection screen as disabled cards marked “Coming soon”, which communicates the intended extension point without implying functionality that does not exist. Within vancomycin, the three sampling workflows named in the case study are implemented in full. Optional enhancements described in the case study — camera capture with OCR, what-if simulation, additional medication modules and export or sharing — were not implemented; the corresponding user-interface affordances that do appear are identified in Section 17.5 as non-functional placeholders.

1.6 Structure of This Report

Section 2 states the problem and Section 3 the proposed solution. Section 4 gives a system overview together with a traceability matrix against the case study's mandatory requirements, and Section 5 documents the technology stack. Sections 6 to 8 present the static design — architecture, use case model and component structure. Sections 9 to 11 present the dynamic design through activity, sequence and data-flow models. Sections 12 to 15 cover navigation, the domain class model,

validation and security. Section 16 documents the calculation engine and three worked examples. Section 17 documents the user interface with screenshots. Sections 18 to 20 cover testing, version control and responsible use of artificial intelligence. Sections 21 and 22 state the limitations and conclusion.

2. Problem Statement

2.1 The Clinical Problem

Vancomycin has a narrow therapeutic index. Sub-therapeutic exposure permits the MRSA infection to progress and encourages resistance; supra-therapeutic exposure is associated with acute kidney injury. Because clearance of the drug depends principally on renal function, and because renal function varies enormously between an eighteen-year-old trauma patient and a seventy-eight-year-old with stage 4 chronic kidney disease, a fixed milligram-per-kilogram dose is not adequate. Each patient needs an individualised regimen derived from their own measured concentrations.

2.2 Limitations of Current Practice

In the hospital pharmacy setting described in the case study, these calculations are still performed by hand, on paper worksheets or with a general-purpose calculator. That practice has four specific weaknesses.

- **Method selection is left to memory.** Which set of equations applies depends on which samples the laboratory returned. Selecting the trough-only pathway when a peak is also available discards information; selecting a two-point pathway when only one sample exists is not possible at all. Nothing in a paper worksheet enforces the correct choice.
- **There is no input validation.** A worksheet accepts a serum creatinine of 9.8 $\mu\text{mol/L}$ as readily as 98 $\mu\text{mol/L}$. A transposed decimal point propagates silently into the dose recommendation.
- **Cross-field inconsistencies go undetected.** If the trough is recorded as having been drawn before the peak, the two-point elimination-rate calculation produces a negative rate constant — a physically meaningless result that a calculator will nonetheless display.
- **There is no audit trail.** Paper worksheets are discarded, so neither the pharmacist nor the department can review how a past recommendation was reached.

2.3 Who Is Affected

The pharmacist carries the immediate burden: the calculation is slow, easy to get wrong and difficult to justify afterwards. The patient carries the consequence of an error. The pharmacy student carries a third problem — a worked answer on a whiteboard does not show which equation produced which intermediate value, so the method is hard to learn and harder to check.

2.4 How TDM Insight Addresses the Problem

TDM Insight turns the paper worksheet into a guided five-step wizard. The workflow is chosen explicitly and then determines which fields appear, so the wrong equations cannot be applied to the wrong data. Every field is range-checked against physiological limits as it is typed, and cross-field timing relationships are checked before the engine is ever called. The engine itself refuses to divide by zero or take the logarithm of a non-positive number, returning a specific message instead of a misleading value. Each completed calculation is written to a PostgreSQL table, giving the department the audit trail the paper process lacked. Finally, the Explanation screen reproduces every formula with the patient's own numbers substituted, which converts the tool from a black box into a teaching aid.

3. Proposed Solution

3.1 Solution Overview

TDM Insight is a single-activity Android application built on Jetpack Compose. A user signs in, creates a fictional case, chooses vancomycin, selects one of three sampling workflows, enters the fields that workflow requires, reviews them, and runs the calculation. A pure-Kotlin engine fits a one-compartment pharmacokinetic model to the input and returns a structured result containing the elimination rate constant, half-life, volume of distribution, clearance, projected peak and trough, AUC_{24} and a recommended dose. The result screen presents AUC_{24} on a ring gauge against the 400–600 mg·h/L target band, and the completed case is written asynchronously to a Supabase PostgreSQL database so it appears in the user's history.

3.2 Key Features

- **Three pharmacokinetic pathways.** Pre-dose uses the Cockcroft–Gault creatinine clearance estimate with a population volume of distribution; Post-dose fits the elimination rate constant to a single measured concentration by Newton–Raphson iteration; Pre + Post uses Sawchuk–Zaske two-point log-linear regression to derive both the rate constant and an individualised volume of distribution.
- **Dynamic forms.** `InputFormScreen` renders the pre-dose sample group only for the Pre and Pre + Post workflows and the post-dose group only for Post and Pre + Post, so the user never sees a field that does not apply.
- **Two-tier validation.** Blocking errors are shown in red and disable the Continue button; advisory warnings are shown in amber and do not. The distinction is carried in the type system by `ValidationReport`, which holds separate error and warning lists.
- **Live derived feedback.** The patient-details screen recomputes Cockcroft–Gault creatinine clearance on every keystroke and shows it in a stat chip, so an implausible creatinine value is visible before the user leaves the screen.

- **Explainable output.** A four-phase Explanation screen prints input values, the elimination-rate derivation with the patient's numbers substituted into the formula, the full pharmacokinetic parameter set, and the final AUC₂₄ and dose recommendation.
- **Persistent, per-user case history.** Completed cases are stored in PostgreSQL and the twenty most recent rows for the signed-in user are shown on the Home and History screens.
- **Full light and dark themes.** Two hand-authored Material 3 colour schemes are selectable as Light, Dark or System from Settings.

3.3 Scope Boundaries and Deliberate Extensions

The case study lists user authentication and a cloud backend among the items that are **not required** for the core project (Mohd Zaki, 2026). TDM Insight nevertheless implements both, as optional enhancements layered on top of a complete core.

The justification is the audit-trail requirement identified in Section 2.2. A calculation history is of little value if it is not attributable to a specific user, and the case study does list local calculation history among the permitted optional enhancements. The team chose to satisfy that enhancement with a hosted PostgreSQL table rather than on-device storage, which necessarily brought identity with it.

Critically, the extension does not compromise the core. The calculation engine, the validators, the wizard and the results and explanation screens contain no network code and no authentication dependency. `SupabaseClientProvider.isConfigured` and `AuthRepository.isConfigured` both report false when the corresponding credentials are absent from `local.properties`, and `MainActivity` then bypasses the authentication gate entirely and starts the navigation graph directly. Persistence failures are caught inside `SupabaseRepository` and logged rather than propagated. A grader building the project without any credentials therefore still gets a fully functional TDM calculator; only the history list and the sign-in screen are absent.

4. System Overview and Requirements

4.1 Requirements Traceability

Table 1 maps each mandatory requirement from the case study to the source artefact that satisfies it.

Table 1

Traceability of the case study's mandatory requirements to the implementation

#	Mandatory requirement	Implemented in	Status
1	Native Android, Kotlin, Android Studio, Compose/XML, Material 3	Kotlin_app/ · Compose + Material 3 throughout · build.gradle.kts (compileSdk 35, minSdk	Met

#	Mandatory requirement	Implemented in	Status
		26)	
2	Vancomycin TDM calculation workflows (Pre, Post, Pre + Post)	VancoEngine.calculatePre/calculatePost/calculatePrePost	Met
3	Dynamic input forms driven by the selected method	InputFormScreen — sample groups rendered conditionally on VancoWorkflow	Met
4	Input and cross-field validation	InputValidator (7 public functions) · ValidationReport	Met
5	Calculation engine separated from the user interface	domain/engine/VancoEngine.kt — pure Kotlin, no Android imports	Met
6	Intermediate and final pharmacokinetic results	PkResults — k_e , $t_{1/2}$, V_d , V_d/kg , CL , AUC_{24} , recommended dose, C_{min} , C_{max}	Met
7	Explainable results (inputs → intermediates → parameters → result)	ExplanationScreen — four ExplanationPhase blocks in that exact order	Met
8	Clean Material 3 interface with light and dark mode	ui/theme/Color.kt, Theme.kt · SettingsScreen theme selector	Met
9	Mandatory academic disclaimer	First-launch gate in MainActivity · FictionalDataBanner on input screens · DisclaimerScreen	Met

Note. Requirement numbering follows Section 2 of PROJECT_RULES.md and Sections 2–9 of the case study description. Optional enhancements are addressed separately in Section 21.

4.2 Functional Requirements

Table 2

Functional requirements as implemented

ID	Requirement	Realised by
FR-01	The system shall allow a user to register an account with an email address and password.	SignUpScreen → AuthViewModel.signUp() → Clerk /v1/client/sign_ups
FR-02	The system shall allow a registered user to sign in and shall restore the session on subsequent launches.	LoginScreen · AuthRepository.savedSession() backed by private SharedPreferences
FR-03	The system shall capture patient demographics: case label, weight, height, age, sex and serum creatinine.	NewCaseScreen → PatientInput

ID	Requirement	Realised by
FR-04	The system shall display Cockcroft–Gault creatinine clearance as the demographics are entered.	NewCaseScreen — recomputed in a remember block keyed on s.patient
FR-05	The system shall let the user select the medication to be monitored.	MedicationSelectScreen — vancomycin enabled; gentamicin and amikacin disabled
FR-06	The system shall let the user select one of three sampling workflows.	WorkflowSelectScreen → CaseViewModel.selectWorkflow()
FR-07	The system shall render only the input fields required by the selected workflow.	InputFormScreen — conditional FormGroup blocks
FR-08	The system shall validate every field on change and report blocking errors separately from advisory warnings.	LaunchedEffect → CaseViewModel.validate() → ErrorBanner / WarningBanner
FR-09	The system shall prevent the calculation from running while blocking errors remain.	PrimaryPillButton(enabled = s.validationReport.isValid) and an early return in runCalculation()
FR-10	The system shall present a read-only review of all inputs before the calculation runs.	ReviewScreen
FR-11	The system shall compute k_e , $t_{1/2}$, V_d , V_d/kg , CL , AUC_{24} , recommended dose, C_{min} and C_{max} as applicable to the workflow.	VancoEngine → PkResults
FR-12	The system shall classify AUC_{24} against the 400–600 mg·h/L target and recommend a dose centred on 500 mg·h/L.	ResultBody verdict chip · VancoEngine.recommendedDose()
FR-13	The system shall report a specific, human-readable message when a calculation cannot be completed.	CalculationResult.Failure → ErrorScreen with plain-language and technical lines
FR-14	The system shall present a step-by-step explanation of how the result was obtained.	ExplanationScreen — four phases with formulas and substituted values
FR-15	The system shall persist each successful calculation against the signed-in user.	SupabaseRepository.saveCase() → INSERT INTO cases
FR-16	The system shall list the user's twenty most recent saved cases.	SupabaseRepository.loadRecentCases(20) → HistoryScreen, HomeScreen
FR-17	The system shall let the user choose a light, dark or system-matched theme.	SettingsScreen → UserPrefs.theme → TDMInsightTheme
FR-18	The system shall display the academic-prototype disclaimer on first launch and in the About screen.	FirstLaunchDisclaimer in MainActivity · DisclaimerScreen
FR-19	The system shall allow the user to sign out and clear the stored session.	SettingsScreen → AuthViewModel.signOut() → prefs.edit { clear() }

4.3 Non-Functional Requirements

Table 3

Non-functional requirements and the evidence for each

ID	Requirement	Evidence in the repository
NFR-01	The calculation engine shall contain no Android framework dependency.	VancoEngine.kt imports only kotlin.math and the project's own model and validation packages.
NFR-02	No business rule shall live inside a Composable.	All arithmetic sits in VancoEngine; all rules sit in InputValidator. Composables call the ViewModel only.
NFR-03	Credentials shall never be committed to version control.	local.properties is git-ignored; secrets.defaults.properties is the committed template; values reach the app through BuildConfig.
NFR-04	The interface shall support both light and dark Material 3 colour schemes.	lightColorScheme and darkColorScheme are defined explicitly in Theme.kt from named tokens in Color.kt.
NFR-05	A missing or unreachable backend shall not prevent a calculation.	SupabaseRepository catches every exception and logs it; saveCase() runs in a detached coroutine.
NFR-06	The calculation shall not block the user interface thread.	runCalculation() runs inside viewModelScope.launch; the result is delivered through StateFlow.
NFR-07	Every validation failure shall carry a specific, field-level message.	FieldResult.Error(field, message) — no generic "invalid input" string exists in the codebase.
NFR-08	User input shall survive backward navigation within the wizard.	A single CaseViewModel is scoped to the NavHost and shared by all wizard destinations.
NFR-09	The clinical disclaimer shall be unavoidable on first launch.	MainActivity renders FirstLaunchDisclaimer before any other destination until it is accepted.

5. Technology Stack

Every entry below is present in the project's build configuration. Versions are read from Kotlin_app/gradle/libs.versions.toml and landingPage/package.json.

Table 4

Android application stack

Layer	Technology	Version	Role in TDM Insight
Language	Kotlin	2.0.0	The entire application; the engine is written in

Layer	Technology	Version	Role in TDM Insight
			framework-free Kotlin.
Build	Android Gradle Plugin (Kotlin DSL)	8.5.2	build.gradle.kts; injects credentials into BuildConfig.
Platform	Android SDK	compileSdk 35 · minSdk 26 · targetSdk 35	Android 8.0 and above.
UI toolkit	Jetpack Compose (BOM)	2024.08.00	All eighteen screens are Composables; the project contains no XML layouts.
Design system	Material 3	via Compose BOM	Colour schemes, typography, shapes and components.
Icons	material-icons-extended	via Compose BOM	Outlined and filled icon set used across the UI.
Navigation	Navigation Compose	2.7.7	TdmNavGraph with thirteen routes, one parameterised.
State	Lifecycle ViewModel Compose	2.8.4	AndroidViewModel subclasses exposing StateFlow.
Concurrency	kotlinx-coroutines-android	1.8.1	viewModelScope for the calculation and persistence paths.
Serialisation	kotlinx-serialization-json	1.7.1	CaseDto and the Clerk wire-format models.
HTTP	Ktor client (OkHttp engine)	2.3.12	Clerk Frontend API calls and the Supabase transport.
Testing	JUnit 4	4.13.2	31 unit tests over the engine and validators.
Instrumentation	AndroidX Test · Espresso · Compose UI test	1.2.1 · 3.6.1	Declared dependencies; no instrumented tests are written yet.

Table 5*Backend and external services*

Service	Technology	Role	What ships in the APK
Database	Supabase — hosted PostgreSQL with PostgREST	Two tables (cases, user_profiles), Row Level Security enabled, one trigger.	Project URL and the public anon key only.
Database client	supabase-kt postgrest-kt 2.6.1 (BOM)	Typed insert and select against PostgREST. GoTrue and Realtime are not installed.	—
Identity	Clerk Frontend API, called directly over REST	Email and password sign-up and sign-in; session JWT issuance; optional Supabase token template.	Publishable key (pk_test_ / pk_live_) only.

Service	Technology	Role	What ships in the APK
Session storage	Android private SharedPreferences (tdm_auth)	Persists user id, email, session token and session id across launches.	On-device, application-private.

Table 6*Marketing landing page stack*

Layer	Technology	Version
Framework	React	19.1.0
Build tool	Vite	6.3.5
Language	TypeScript	5.9.3
Styling	Tailwind CSS (Vite plugin) plus a hand-written landing.css	4.1.8
Routing	React Router DOM	7.6.2
Icons	lucide-react	0.511.0
Package manager	Bun	1.3.14

Note. The landing page is a static marketing site. It makes no network request to any backend: ``src/lib/apiClient.ts`` and ``src/lib/useApiQuery.ts`` are unused scaffolding left over from a template and are not reachable from ``App.tsx``. This is represented faithfully in Figure 1.

Table 7*Development and project tooling*

Tool	Use
Android Studio	Primary IDE for the Kotlin application.
Gradle 8.x with the Kotlin DSL	Build, dependency resolution and BuildConfig credential injection.
Git and GitHub	Version control; the repository is the primary submission artefact.
Supabase SQL Editor	Applying supabase/schema.sql and supabase/update_v2.sql.
Clerk Dashboard	Enabling email and password authentication and issuing the publishable key.
JUnit 4 via <code>./gradlew :app:test</code>	Running the engine and validator test suites.
Claude Code (Anthropic)	AI pair-programming assistant; usage is declared in Section 20.

6. System Architecture

6.1 Architectural Pattern and Justification

TDM Insight uses a **layered architecture organised around the Model–View–ViewModel pattern**, which is the architecture Google recommends for modern Android applications (Google, 2024a). Four considerations made it the right choice here.

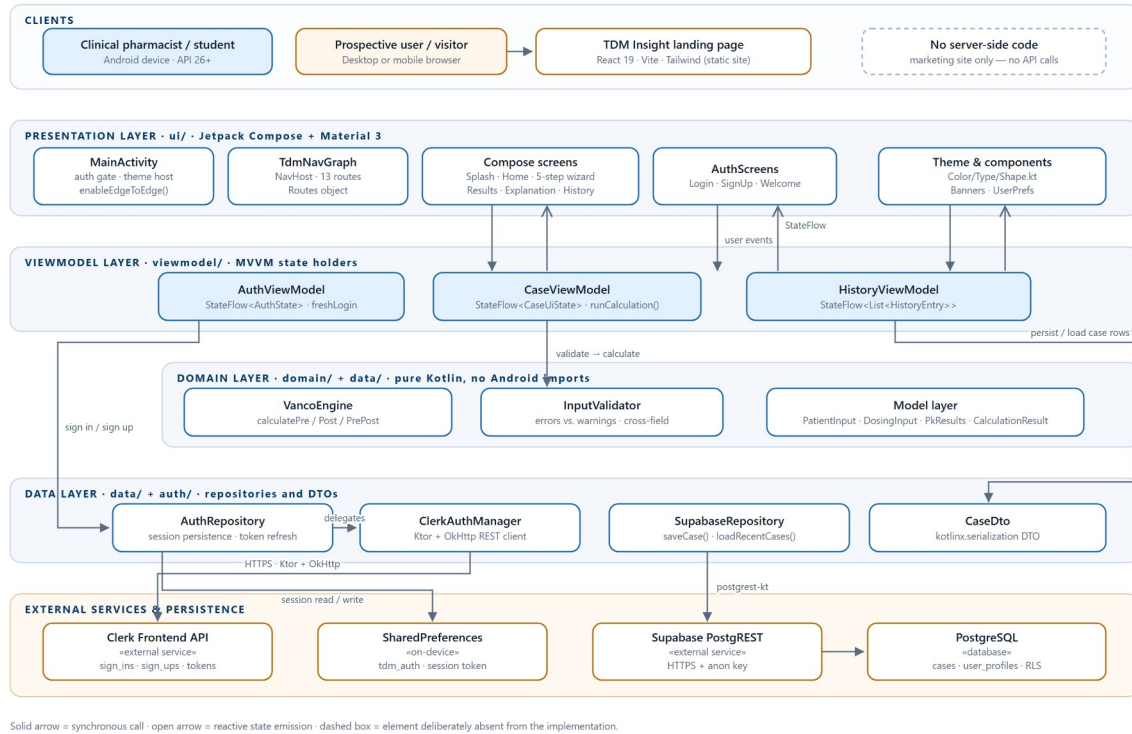
1. **Testability of the engine.** The assessment rubric weights functionality and code quality at forty-five per cent combined, and the case study calls the calculation engine the application's main technical component. Isolating that engine in a layer with no Android dependency means it can be exercised by plain JVM unit tests that run in under a second, without an emulator.
2. **Natural fit with Compose.** A `StateFlow` exposed by a `ViewModel` is consumed by a `Composable` through `collectAsState()`, so recomposition is driven by state change rather than by imperative view updates.
3. **Survival of configuration change.** `ViewModels` outlive activity recreation, so the five-step wizard does not lose the user's input when the device is rotated. This directly satisfies NFR-08.
4. **Avoidance of the “massive activity” anti-pattern.** The application has a single activity of roughly 260 lines whose only responsibilities are theming and the authentication gate.

Dependency injection is performed manually. `TdmApplication` holds four lazily initialised singletons — the Supabase client, the Clerk authentication manager, the authentication repository and the Supabase repository — and `AndroidViewModel` subclasses reach them through the application instance. For four objects this is simpler and more transparent than introducing Hilt or Koin, and it keeps the dependency graph explicit enough for a student to defend in a viva.

6.2 The Architecture in Detail

Figure 1

Layered system architecture of TDM Insight



Note. Solid arrows are synchronous calls; open arrows are reactive state emissions. The dashed box records that the landing page has no server-side component. Derived from the Android source tree and 'landingPage/src/'.

Table 8

Responsibilities of each architectural layer

Layer	Package	Responsibility	Must not
Presentation	ui.screens, ui.navigation, ui.theme, ui.components	Render state, collect user input, navigate.	Contain arithmetic, validation rules or network calls.
ViewModel	viewmodel	Hold immutable UI state, orchestrate calls into the domain and data layers, expose StateFlow.	Contain pharmacokinetic formulas.
Domain	domain.engine, data.model, data.validation	Validate structured input and compute pharmacokinetic parameters.	Import anything from the Android framework.
Data	data.supabase, auth	Map domain objects to and from wire formats; own all network and persistence access.	Decide business rules or format values for display.

6.3 Layer Interaction

The interaction is strictly downward for calls and upward for state. HomeScreen collects `HistoryViewModel.entries`; `HistoryViewModel` calls `SupabaseRepository.loadRecentCases()`; the repository calls PostgREST. Nothing in the domain layer knows that a repository exists, and nothing in the repository layer knows that Compose exists. The one architectural nuance worth noting is that `HistoryEntry`, the display model for a saved case, lives in `ui.screens` and is marked `internal`; `SupabaseModels.kt` therefore contains a `toHistoryEntry()` mapper that converts the transport DTO into that display model at the data-layer boundary.

7. Use Case Model

7.1 Actors

Table 9

System actors

Actor	Type	Description
Clinical pharmacist / pharmacy student	Primary human actor	Registers, signs in, creates fictional cases, selects a workflow, enters and reviews inputs, runs calculations, reads results and explanations, browses history and configures the application.
Prospective user	Secondary human actor	Reaches the project through the public landing page. Has no account and does not interact with the Android application.
Clerk	External system actor	Authenticates credentials and issues the session JWT. Reached over HTTPS from <code>ClerkAuthManager</code> using the Ktor client.
Supabase	External system actor	Stores completed cases and returns the user's recent case rows. Reached through PostgREST by the <code>postgrest-kt</code> client.

7.2 Use Case Summary

Table 10

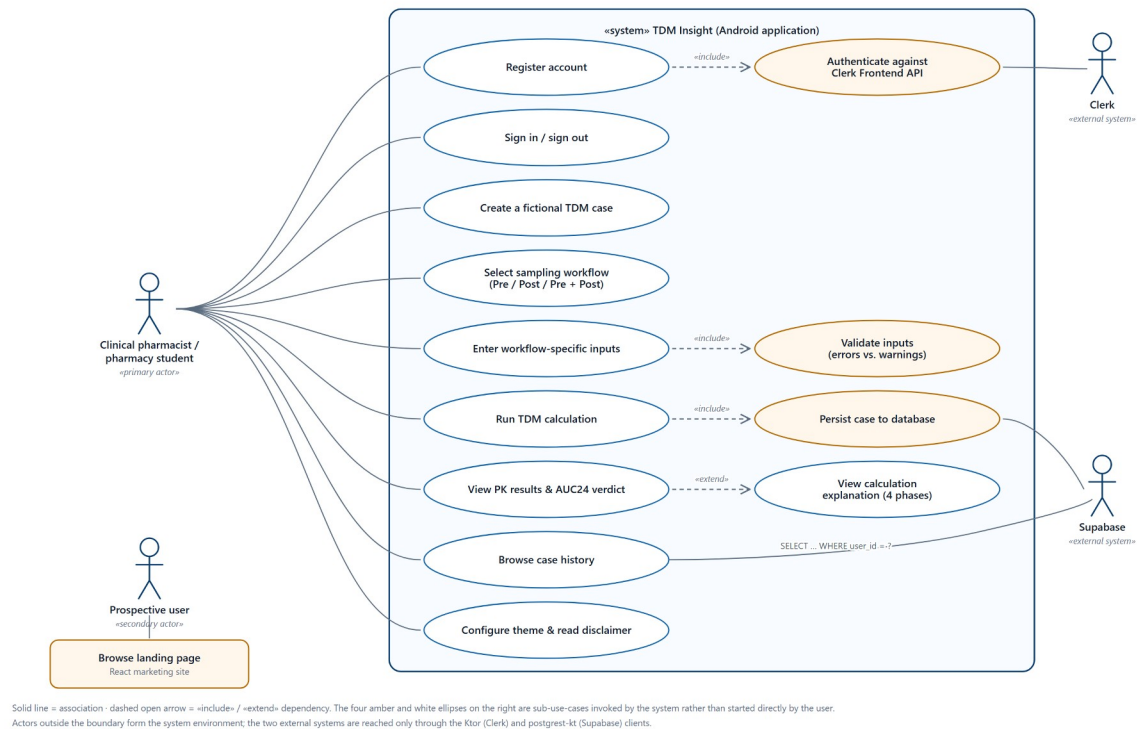
Use case summary

ID	Use case	Primary actor	Entry point in the code
UC-01	Register account	Pharmacist	<code>SignUpScreen</code>
UC-02	Sign in / sign out	Pharmacist	<code>LoginScreen</code> ,

ID	Use case	Primary actor	Entry point in the code
			SettingsScreen
UC-03	Create a fictional TDM case	Pharmacist	NewCaseScreen
UC-04	Select sampling workflow	Pharmacist	WorkflowSelectScreen
UC-05	Enter workflow-specific inputs	Pharmacist	InputFormScreen
UC-06	Run TDM calculation	Pharmacist	ReviewScreen → CaseViewModel.runCalculation()
UC-07	View PK results and AUC ₂₄ verdict	Pharmacist	ResultsScreen
UC-08	View calculation explanation	Pharmacist	ExplanationScreen
UC-09	Browse case history	Pharmacist	HistoryScreen
UC-10	Configure theme and read the disclaimer	Pharmacist	SettingsScreen, DisclaimerScreen
UC-11	Authenticate against Clerk	Clerk («include» of UC-01, UC-02)	ClerkAuthManager.signIn/signup
UC-12	Validate inputs	System («include» of UC-05)	InputValidator.merge(...)
UC-13	Persist case to database	Supabase («include» of UC-06)	SupabaseRepository.saveCase()

Figure 2

UML use case diagram for TDM Insight



Note. Amber ellipses are sub-use-cases invoked by the system rather than started directly by the user. Actors outside the boundary constitute the system environment.

7.3 Detailed Use Case Specification

Table 11

Use case specification — UC-06 Run TDM Calculation

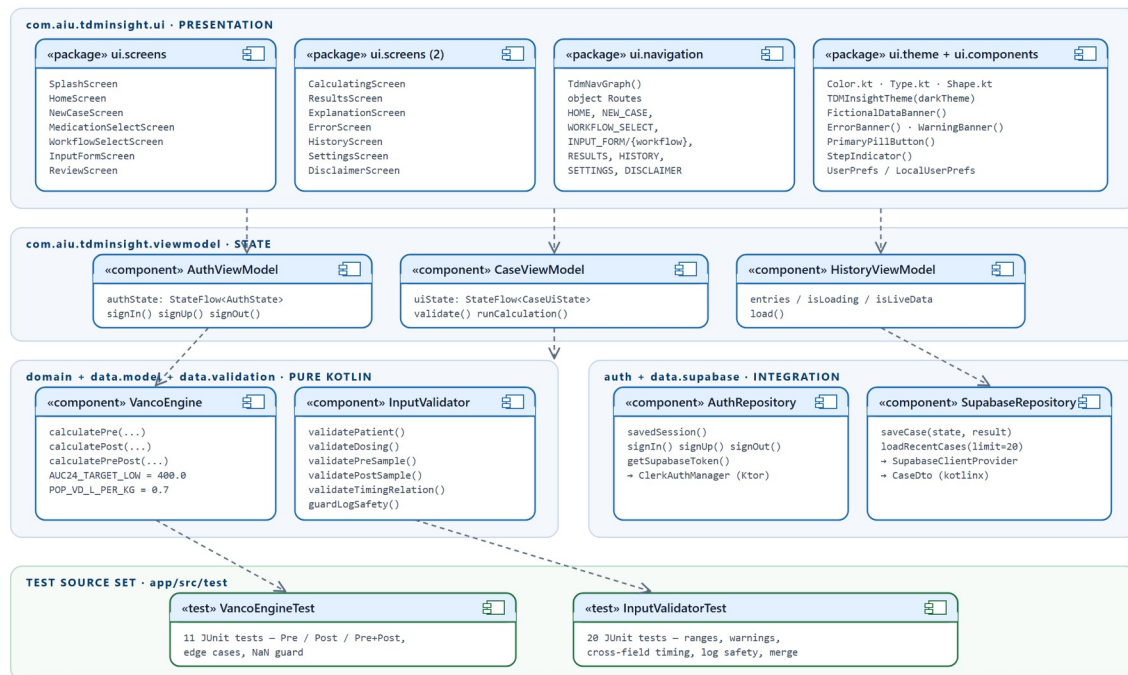
Field	Content
Identifier	UC-06
Primary actor	Clinical pharmacist / pharmacy student
Secondary actors	Supabase (through the «include» relationship to UC-13)
Goal	Obtain AUC_{24} , the full pharmacokinetic parameter set and a dose recommendation for a fictional case.
Preconditions	A workflow has been selected; all fields that workflow requires have been entered; <code>ValidationReport.isValid</code> is true, so the Review screen's Run control is enabled.
Main success scenario	1. The pharmacist reviews the summarised inputs on ReviewScreen. 2. The pharmacist taps Run calculation. 3. <code>CaseViewModel.runCalculation()</code> re-runs <code>validate()</code> as a second line of defence. 4. The matching <code>VancoEngine</code> function is invoked inside <code>viewModelScope</code>. 5. The engine validates again, fits the model and returns <code>CalculationResult.Success(PkResults)</code>. 6. <code>CalculatingScreen</code> observes the new state and navigates to ResultsScreen. 7. AUC_{24} is drawn on the ring gauge with an in-target or out-of-target verdict chip. 8. In a detached coroutine, <code>SupabaseRepository.saveCase()</code> inserts the row (UC-13).
Alternative flow A — validation fails	At step 3, if <code>report.isValid</code> is false the method returns immediately. No engine call is made and the error banner remains visible on the input form.
Alternative flow B — engine failure	At step 5, if a guard trips the engine returns <code>CalculationResult.Failure(message)</code> . <code>CalculatingScreen</code> navigates to ErrorScreen, which shows a plain-language line and the technical message.
Alternative flow C — persistence failure	At step 8, any exception is caught inside <code>SupabaseRepository</code> and logged at warning level. The displayed result is unaffected; the case simply does not appear in history.
Postconditions	<code>CaseUiState.result</code> holds a Success; on the success path a new row exists in the cases table, and the <code>trg_ensure_user_profile</code> trigger has created a <code>user_profiles</code> row if none existed.

8. Component and Package Structure

The main source set contains twenty-five Kotlin files under `com.aiu.tdm Insight`, grouped into six top-level areas — `ui`, `viewModel`, `domain`, `data`, `auth` and the application root — plus two test classes. Figure 3 shows the components and their «use» dependencies, including the test source set.

Figure 3

Component and package structure



Note. Dashed open arrows are «use» dependencies. No arrow enters the domain layer from the data or presentation layers, which is what makes the engine testable in isolation.

The dependency direction visible in Figure 3 is the architectural invariant of the project. `VancoEngine` and `InputValidator` depend only on `data.model`, and both are depended upon by `CaseViewModel` and by the test source set. Neither depends on `ui`, `auth` or `data.supabase`. Removing the entire networking stack from the project would leave the calculation path compiling and its tests passing.

9. Behavioural Models

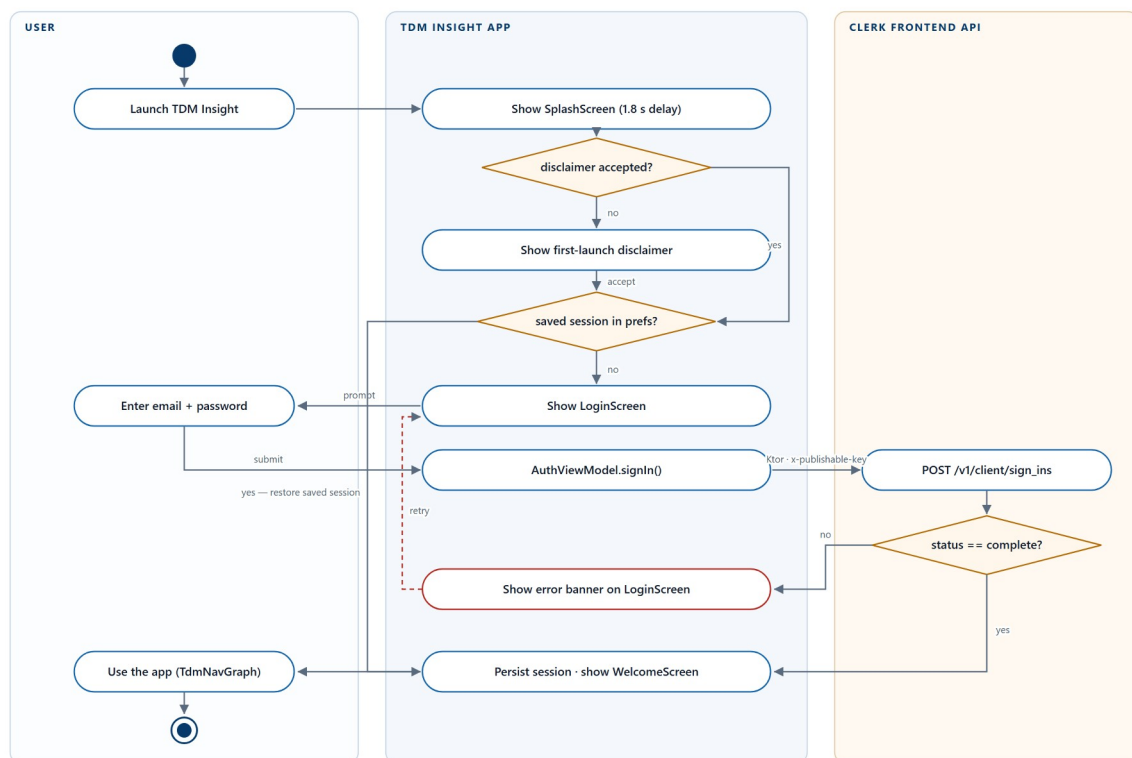
9.1 Activity Diagram — Application Entry and Authentication

`MainActivity` implements a gate with eight ordered branches. The splash screen is always shown first for 1.8 seconds. The first-launch disclaimer is then shown until it is accepted. If Clerk is not configured the gate is skipped altogether and the navigation graph starts. Otherwise a saved session

is restored from SharedPreferences if one exists, and a login or sign-up screen is presented if not. A fresh sign-in or sign-up shows the welcome screen exactly once, because AuthViewModel sets a freshLogin flag that a session restore does not set and that the welcome screen's Continue button consumes.

Figure 4

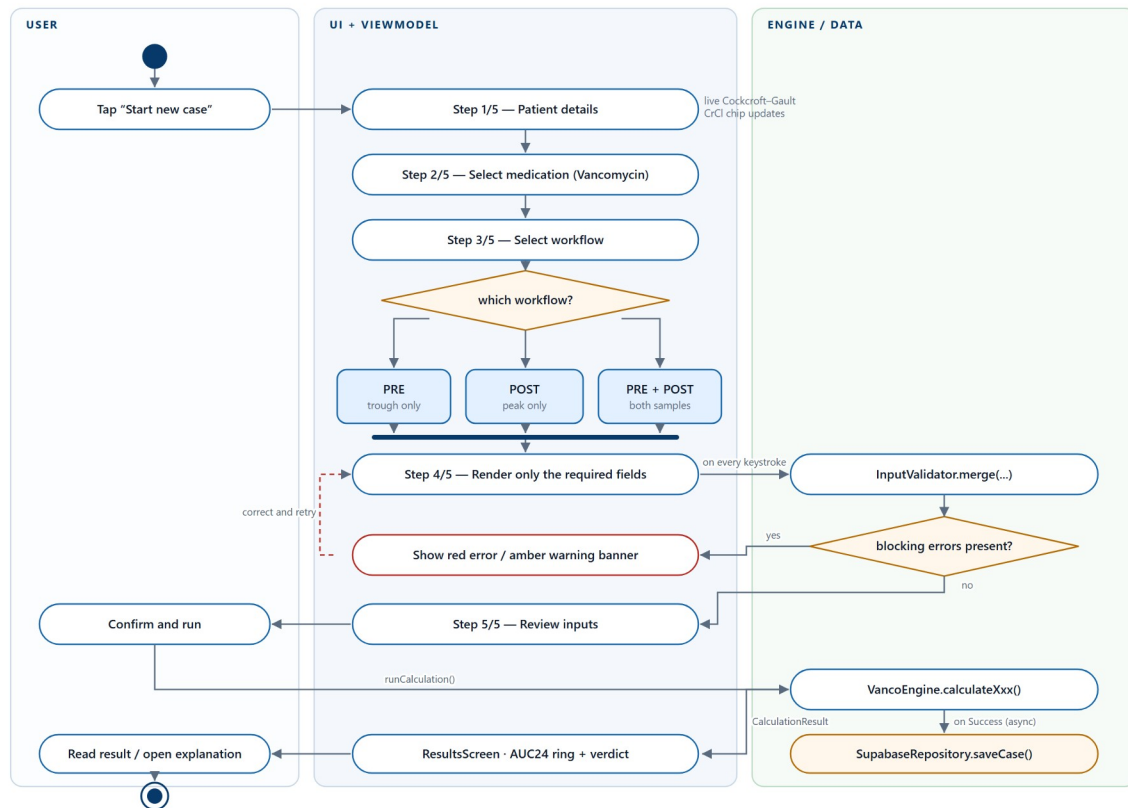
Activity diagram — application entry and authentication



Note. Traced from the `when` block in `MainActivity.onCreate`, `AuthViewModel`, `AuthRepository` and `ClerkAuthManager.signIn`.

9.2 Activity Diagram — The Five-Step Calculation Workflow

The calculation wizard is the application's principal flow. Its most important structural property is the decision node after step three: the selected workflow determines which field groups step four renders, and therefore which validators run and which engine function is ultimately called. Validation is not a gate at the end of the form; it re-runs on every state change through a LaunchedEffect keyed on the patient, dosing and sample state objects.

Figure 5*Activity diagram — the five-step TDM calculation workflow*

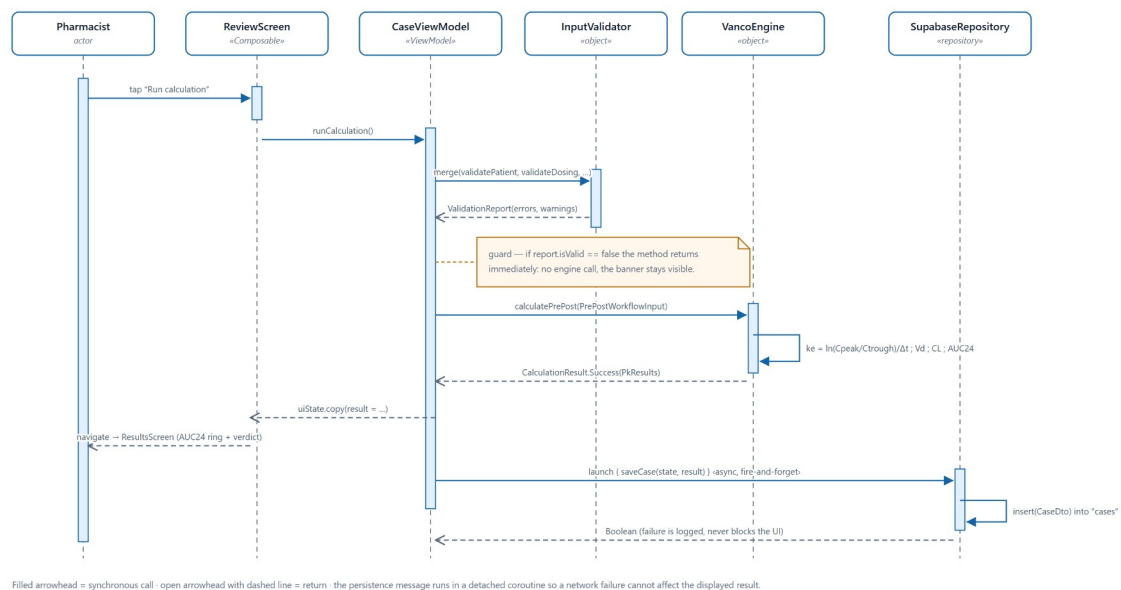
Note. The bar beneath the three workflow branches is a UML merge node: the branches are mutually exclusive and converge on a single input form whose content differs by workflow.

9.3 Sequence Diagram — Running and Persisting a Calculation

Figure 6 traces a Pre + Post calculation from the user's tap to the database write. Two design decisions are visible in the message ordering. First, validation happens **before** the engine is called and again inside it, so the engine is never asked to operate on input the user interface has not already cleared. Second, persistence is dispatched in a nested `launch` after the result has already been written to `uiState`, so the user sees the AUC₂₄ ring without waiting for the network and a failed insert cannot alter what is displayed.

Figure 6

Sequence diagram — running and persisting a Pre + Post calculation

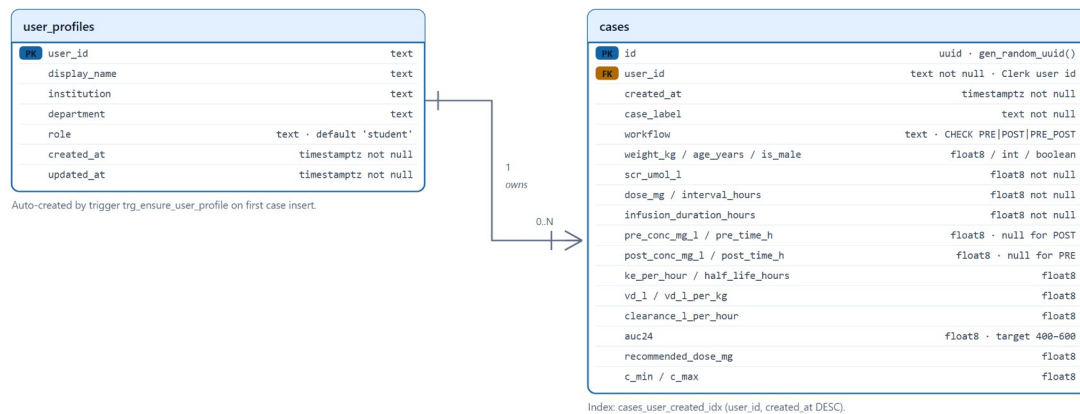


Note. Filled arrowheads are synchronous calls; open arrowheads on dashed lines are returns. Traced from `CaseViewModel.runCalculation()`, `VancoEngine.calculatePrePost()` and `SupabaseRepository.saveCase()`.

10. Data and Database Architecture

10.1 Entity Relationship Model

The schema is deliberately small: two tables, one index, one trigger and five Row Level Security policies. Its shape follows from the domain. A completed calculation is a fact about a moment in time and is never edited, so cases is an append-only table that stores the inputs and the derived parameters side by side. Storing the derived values rather than recomputing them means the history list can be rendered without re-running the engine, and it preserves the audit record even if the engine's constants are later revised.

Figure 7*Entity relationship diagram and access control***ACCESS CONTROL AS IMPLEMENTED (supabase/schema.sql · update_v2.sql)**

- Row Level Security is ENABLED on both tables.
- Active policies are the permissive fallback set — `anon_read_all` (SELECT using true) and `anon_insert` (INSERT with check true).
- Per-user server-side policies keyed on the Clerk JWT 'sub' claim are written but left commented out in `update_v2.sql`, pending a Clerk JWT template.
- Isolation is therefore enforced at query level: `SupabaseRepository.loadRecentCases()` always applies `eq("user_id", userId)`.
- Only the Supabase anon key ships in the APK; the service_role key is never present in the client.

The application performs INSERT and SELECT only. There is no UPDATE or DELETE path from the Android client, so a saved case is an append-only audit record.

Note. Columns, types, constraints and policies are taken verbatim from ``Kotlin_app/supabase/schema.sql`` and ``update_v2.sql``.

10.2 Table Specifications

Table 12*The `cases` table*

Group	Columns	Type and constraint	Notes
Identity	<code>id</code>	uuid, primary key, <code>gen_random_uuid()</code>	Assigned by PostgreSQL; the client never supplies it.
	<code>user_id</code>	text not null, default 'anonymous'	The Clerk user id. Falls back to 'anonymous' when no session exists.
	<code>created_at</code>	timestampz not null default <code>now()</code>	Server clock; used for ordering the history list.
Case	<code>case_label</code>	text not null	The user's fictional case identifier, or a generated Case- <code><epoch></code> label if left blank.
	<code>workflow</code>	text not null, CHECK IN ('PRE', 'POST', 'PRE_POST')	Mirrors the <code>VancoWorkflow</code> enumeration.
Demographics	<code>weight_kg</code> , <code>age_years</code> , <code>is_male</code> , <code>scr_umol_l</code>	float8, int, boolean, float8 — all not null	Height is captured by the UI but is not persisted.
Regimen	<code>dose_mg</code> , <code>interval_hours</code> , <code>infusion_duration_hours</code>	float8 not null	The regimen as prescribed.
Samples	<code>pre_conc_mg_l</code> , <code>pre_time_h</code>	float8, nullable	Null for the POST workflow.

Group	Columns	Type and constraint	Notes
Derived	post_conc_mg_l, post_time_h	float8, nullable	Null for the PRE workflow.
	ke_per_hour, half_life_hours, vd_l, vd_l_per_kg, clearance_l_per_hour	float8, nullable	The intermediate pharmacokinetic parameters.
	auc24, recommended_dose_mg	float8, nullable	The primary outputs. Target band 400–600 mg·h/L.
	c_min, c_max	float8, nullable	c_max is null for the PRE workflow, which returns no projected peak.

Note. Index `cases\_user\_created\_idx (user\_id, created\_at DESC)` supports the history query directly.

Table 13

The `user\_profiles` table

Column	Type	Notes
user_id	text, primary key	The Clerk user id; the join key to cases.
display_name	text, nullable	Not written by the application.
institution	text, nullable	Not written by the application.
department	text, nullable	Not written by the application.
role	text, default 'student'	Intended values: student, pharmacist, doctor.
created_at, updated_at	timestampz not null default now()	Set by the trigger on first insert.

Note. The Android client never writes to this table directly. Rows are created by the `ensure\_user\_profile()` trigger, which fires after each insert into `cases` and performs an `ON CONFLICT DO NOTHING` insert of a minimal profile. The remaining columns are provisioned for a future profile screen.

10.3 Access Control

Row Level Security is enabled on both tables. The policies that are currently active are the permissive fallback pair defined in `update_v2.sql`: `anon_read_all`, a SELECT policy using `(true)`, and `anon_insert`, an INSERT policy with `check (true)`. The per-user policies that key on the Clerk subject claim — `user_id = current_setting('request.jwt.claims', true)::json->>'sub'` — are written into the same migration but are left commented out, because they require a JWT template named `supabase` to be configured in the Clerk dashboard.

The practical consequence must be stated plainly. Data isolation between users is presently enforced at the query level rather than by the database: `loadRecentCases()` always applies `eq("user_id", userId)`, so a user sees only their own cases through the application. It is not enforced against a caller who holds the anon key and issues their own PostgREST request. This is acceptable for an academic prototype that stores only fictional data, and it is recorded as a known limitation in Section 21. Enabling the commented policies is the single step that closes it.

Two properties of the design do hold unconditionally. Only the Supabase *anon* key is compiled into the application; the *service_role* key, which bypasses Row Level Security, is never present in the client. And the application issues only *INSERT* and *SELECT*; there is no code path from the Android client that can update or delete a saved case, which is what makes the history an audit record rather than a mutable list.

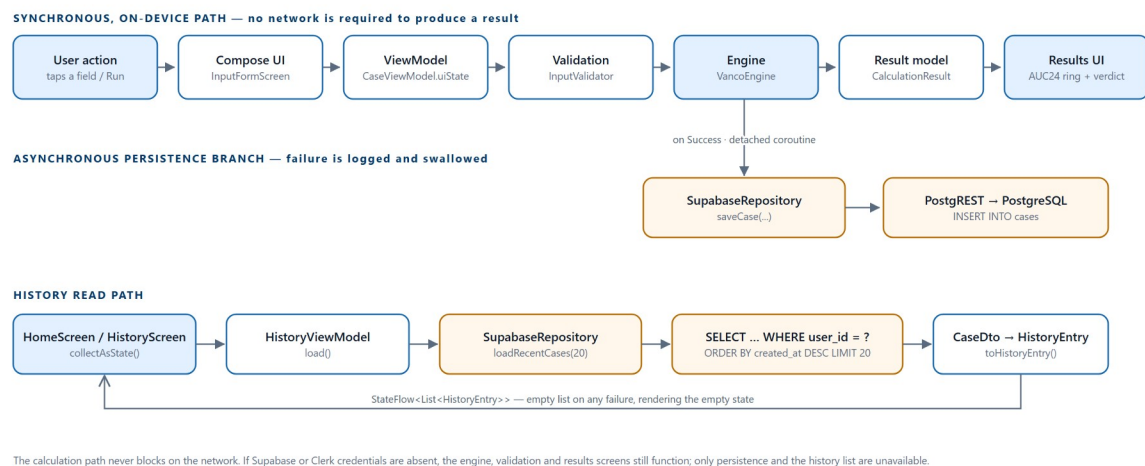
11. Frontend, Backend and Data Flow

Three distinct paths carry data through the system, and it is worth separating them because they have different reliability characteristics.

- **The calculation path is synchronous and entirely on-device.** A user action updates *CaseUiState*, validation runs, the engine computes, and the result flows back to the Results screen. No network call participates. This is the path that must never fail.
- **The persistence path is asynchronous and best-effort.** It branches off the calculation path only on success, runs in a detached coroutine, and swallows its own failures.
- **The history path is a read.** *HistoryViewModel.load()* asks the repository for the user's twenty most recent rows; any failure yields an empty list, which the UI renders as an empty state rather than an error.

Figure 8

End-to-end data flow



Note. Paths traced through `'CaseViewModel'`, `'VancoEngine'`, `'SupabaseRepository'` and `'HistoryViewModel'`.

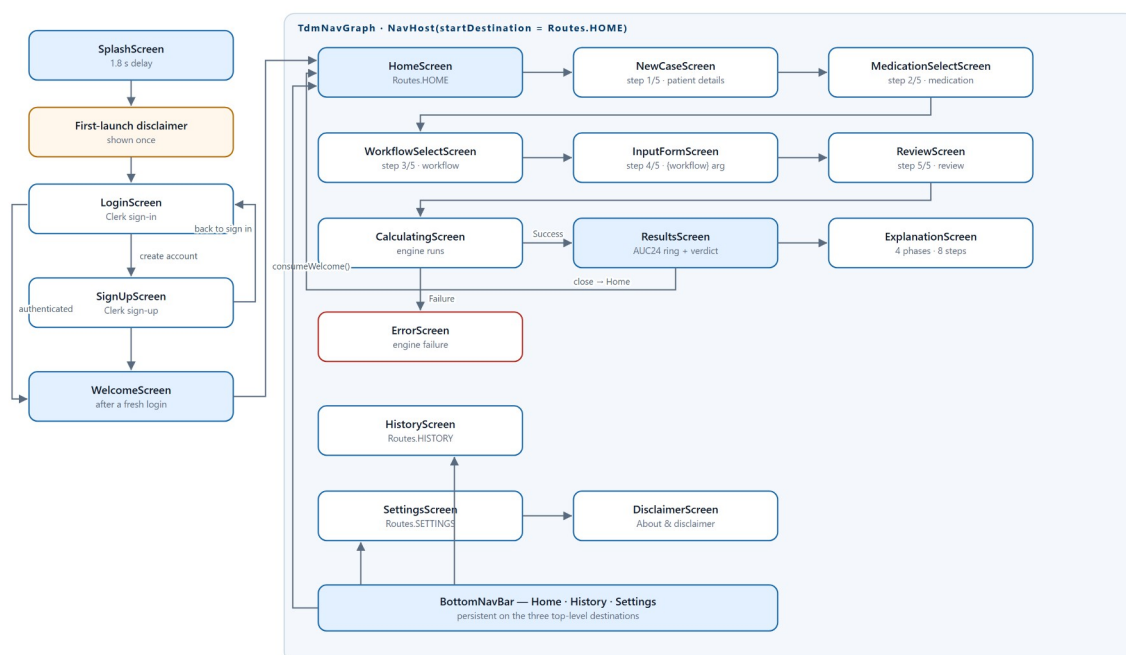
Expressed as the case study expresses it, the core loop is *UI* → *Input State* → *Validation* → *TDM Calculation Engine* → *Result Model* → *Results UI*, with a database write hanging off the end of it. The important architectural point is that the arrow into the database is one-way and optional.

12. Navigation and Screen Map

The application has eighteen distinct screens. Five of them — splash, first-launch disclaimer, login, sign-up and welcome — are hosted directly by MainActivity and sit outside the navigation graph, because they are gates rather than destinations. The remaining thirteen are registered as routes in TdmNavGraph, whose start destination is Routes.HOME. Only one route is parameterised: `input_form/{workflow}`, which carries the lower-cased workflow name as a `NavType.StringType` argument.

Figure 9

Screen map and navigation graph



Screens in the left column are hosted directly by MainActivity and lie outside the navigation graph; the authentication gate decides which of them is shown. Every route name is defined in the Routes object in NavGraph.kt. InputFormScreen is the only parameterised route: "input_form/{workflow}"; where {workflow} is pre, post or pre_post.

Note. Routes and transitions taken from `NavGraph.kt` and the `navigate()` calls in `Screens.kt`.

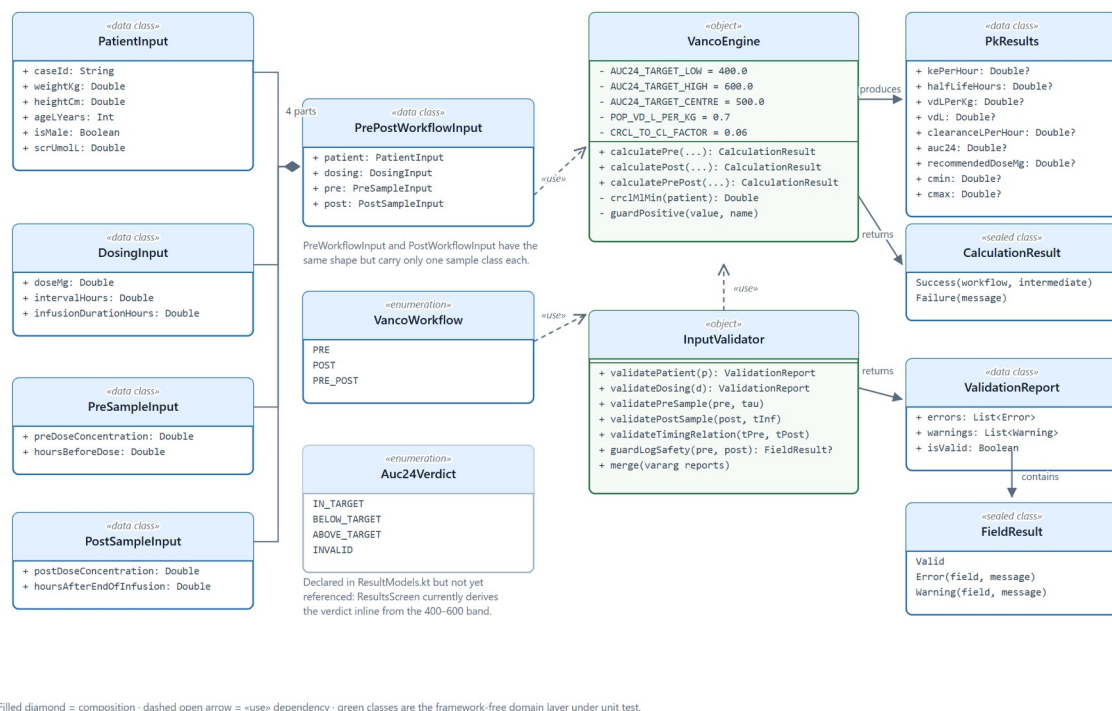
Two navigation behaviours are worth drawing out. CalculatingScreen has no user control at all: a `LaunchedEffect` keyed on `s.result` navigates to ResultsScreen on success or ErrorScreen on failure, in both cases with `popUpTo(Routes.CALCULATING) { inclusive = true }` so the transient screen cannot be reached by pressing back. And the shared `CaseViewModel` is obtained once inside TdmNavGraph and passed to every wizard destination, which is what allows the user to navigate backwards through the five steps without losing entered values.

13. Domain Class Model

The domain model is expressed almost entirely in immutable Kotlin data classes, with sealed classes used wherever a value can be one of a fixed set of shapes. This is a deliberate use of the type system to prevent whole categories of error: because `CalculationResult` is a sealed class with exactly `Success` and `Failure` variants, the compiler forces every caller to handle failure, and no null check or exception handler is needed at the call site.

Figure 10

Domain class diagram



Note. Classes, members and constants are taken verbatim from ``data/model``, ``data/validation`` and ``domain/engine``. The grey enumeration ``Auc24Verdict`` is declared but not yet referenced; ``ResultsScreen`` currently derives the verdict inline from the 400–600 band.

Every field of `PkResults` is nullable, which is not laziness but a faithful representation of the domain: the Pre workflow produces no projected peak, so `cmax` is genuinely absent rather than zero. The Results screen renders an em dash for any null parameter instead of a misleading numeral.

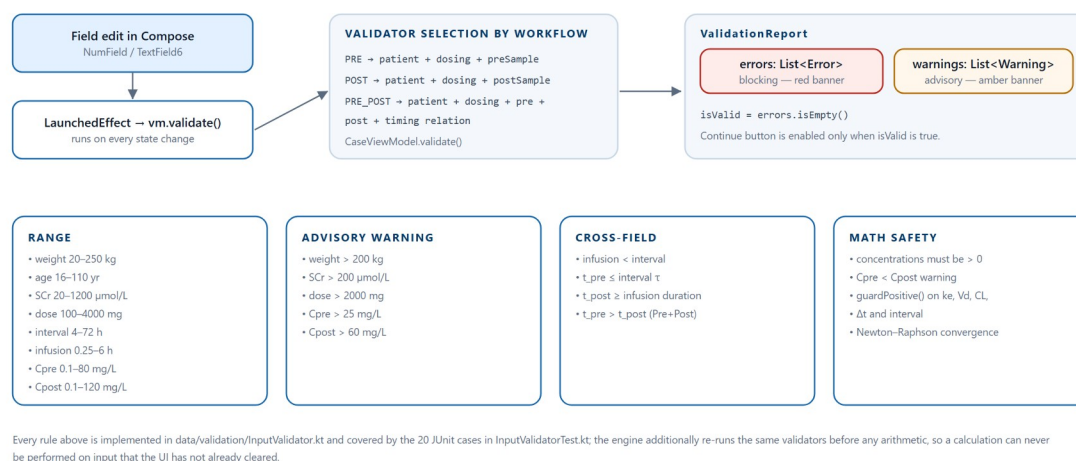
14. Input Validation

The case study is explicit that validation must go beyond checking whether a field is empty, and that the user interface must distinguish an error from information that merely requires review. TDM Insight satisfies both by making the distinction part of the return type rather than a rendering

decision: `InputValidator` returns a `ValidationReport` holding a list of `FieldResult.Error` and a separate list of `FieldResult.Warning`, and `isValid` is defined as `errors.isEmpty()`. Warnings therefore cannot accidentally block a calculation, and errors cannot accidentally be rendered as advisory.

Figure 11

Two-tier validation model



Note. Thresholds read directly from ``data/validation/InputValidator.kt``.

Table 14

Validation rules as implemented

Category	Rule	Outcome
Range	Weight outside 20–250 kg	Error
Range	Age outside 16–110 years	Error
Range	Serum creatinine outside 20–1200 µmol/L	Error
Range	Dose outside 100–4000 mg	Error
Range	Dosing interval outside 4–72 h	Error
Range	Infusion duration outside 0.25–6 h	Error
Range	Pre-dose concentration outside 0.1–80 mg/L	Error
Range	Post-dose concentration outside 0.1–120 mg/L	Error
Advisory	Weight above 200 kg	Warning — “please review”
Advisory	Serum creatinine above 200 µmol/L	Warning — marked renal impairment
Advisory	Single dose above 2000 mg	Warning — unusually high
Advisory	Trough above 25 mg/L	Warning — above the usual 10–20 mg/L range
Advisory	Peak above 60 mg/L	Warning — verify the assay result

Category	Rule	Outcome
Cross-field	Infusion duration is not shorter than the dosing interval	Error
Cross-field	Pre-dose sample time exceeds the dosing interval τ	Error
Cross-field	Post-dose sample time falls inside the infusion	Error
Cross-field	In Pre + Post, the pre-dose sample is not drawn later than the post-dose sample	Error
Math safety	Either concentration is not strictly positive	Error — protects $\ln()$
Math safety	Pre-dose concentration is not less than post-dose	Warning — result may be unreliable
Math safety	Any derived k_e , V_d , CL , Δt or interval is non-finite or non-positive	ArithmeticException caught → Failure
Math safety	Newton–Raphson does not converge within 50 iterations	Failure with a specific message

The cross-field timing rule deserves comment because it is the one most often got wrong on paper. Both sample times are expressed as hours after the start of the same dose. In a Pre + Post case the peak is drawn early in the interval and the trough late, so t_{pre} must be greater than t_{post} . If the pharmacist transposes them, `validateTimingRelation` blocks the calculation with the message “Pre-dose sample must be drawn later in the interval than the post-dose sample”, rather than allowing the engine to produce a negative elimination rate constant.

15. Authentication, Session Management and Security

15.1 Registration and Sign-In

Authentication is implemented against Clerk's Frontend API over plain REST rather than through the Clerk Android SDK. `ClerkAuthManager` derives the Frontend API host by base64-decoding the portion of the publishable key that follows the `pk_test_` or `pk_live_` prefix, then posts a form body to `/v1/client/sign_ins` or `/v1/client/sign_ups` with an `x-publishable-key` header. A response whose status is `complete` yields the session JWT, the user id, the session id and the primary email address; any other status, and any `ClientRequestException`, is converted into a `ClerkResult.Failure` carrying Clerk's own `long_message` where one is available, so the user sees the service's wording rather than a generic string.

The choice to call REST directly rather than adopt the SDK was made so that every network interaction is visible in the project's own source. That is a defensible trade-off for an assessed academic project in which each student must be able to explain the authentication flow in a technical viva, and it avoids a dependency whose behaviour would otherwise be opaque.

15.2 Session Persistence

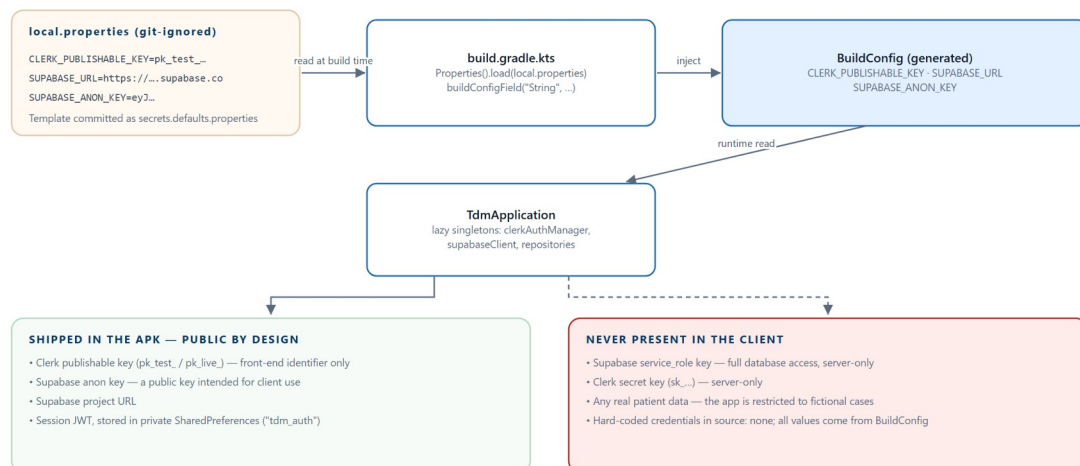
AuthRepository writes the user id, email, session token and session id into private SharedPreferences under the name `tdm_auth`. `savedSession()` reconstitutes an `AuthState.Authenticated` from those four values on the next launch, which is what allows `MainActivity` to skip the login screen. `signOut()` clears the whole preference file. `getSupabaseToken()` first attempts a refresh through Clerk's `/tokens/supabase` template endpoint and falls back to the stored token; this is the hook that would supply the JWT once per-user Row Level Security is enabled.

15.3 Credential Management

No credential is committed to the repository. Three values — the Clerk publishable key, the Supabase project URL and the Supabase anon key — are read from a git-ignored `local.properties` at build time by `build.gradle.kts` and injected as `buildConfigField` entries. `secrets.defaults.properties` is committed as a template with placeholder values so a new developer knows exactly which keys to supply.

Figure 12

Credential and security model



Note. Traced from `'build.gradle.kts'`, `'gitignore'`, `'secrets.defaults.properties'` and `'TdmApplication.kt'`.

All three injected values are public by design — a Clerk publishable key and a Supabase anon key are both intended to ship inside client applications — so their presence in the APK is not a leak. The keys that would be a leak, the Supabase `service_role` key and the Clerk secret key, are never referenced anywhere in the project. The application declares a single Android permission, `INTERNET`; it requests no access to storage, location, camera or contacts.

16. The Calculation Engine

VancoEngine is a Kotlin object of 276 lines. It declares five named constants, each carrying a source comment, and exposes three public functions returning CalculationResult. Every function follows the same shape: validate, compute inside a try block, and convert any ArithmeticException raised by a guard into a Failure.

16.1 Named Constants

```
private const val AUC24_TARGET_LOW      = 400.0    // Rybak et al. (2020)
private const val AUC24_TARGET_HIGH    = 600.0    // Rybak et al. (2020)
private const val AUC24_TARGET_CENTRE  = 500.0    // dose recommendation target
private const val POP_VD_L_PER_KG      = 0.7      // population prior, single-point
methods
private const val CRCL_TO_CL_FACTOR    = 0.06     // CrCl (mL/min) → CL (L/h)
```

Clinical constants are named and sourced rather than embedded as literals, as required by Section 7 of PROJECT_RULES.md.

16.2 Shared Relationships

Three relationships are used by all three workflows.

```
CrCl (mL/min) = ((140 - age) × weight × sexFactor) / (72 × SCr_mg/dL)
                where SCr_mg/dL = SCr_μmol/L ÷ 88.4 and sexFactor = 1.0 male, 0.85
                female

AUC24 (mg·h/L) = (Dose ÷ τ) × 24 ÷ CL

Recommended dose (mg) = (AUC_target × CL × τ) ÷ 24, with AUC_target = 500 mg·h/L
```

The creatinine clearance relationship is the Cockcroft–Gault equation (Cockcroft & Gault, 1976). The divisor 88.4 converts micromoles per litre to milligrams per decilitre, which is the unit the original equation was published in; the application collects the value in micromoles per litre because that is the unit Malaysian laboratories report.

16.3 The Pre-Dose Workflow

When only a trough is available there is not enough information to fit an individualised volume of distribution, so the engine uses a population prior of 0.7 L/kg and estimates clearance from renal function.

```
Vd = 0.7 × weight
CL = CrCl × 0.06
ke = CL ÷ Vd
t½ = ln(2) ÷ ke
AUC24, recommended dose as above
```

The measured trough is returned unchanged as cmin; no projected peak is produced, so cmax is null. This is the least individualised of the three pathways and the report notes it as such: its accuracy depends entirely on how well the population priors describe the particular patient.

16.4 The Post-Dose Workflow

With a single post-infusion concentration the engine fits the elimination rate constant to a one-compartment constant-rate infusion model by Newton–Raphson iteration (Burden & Faires, 2010), again using the population volume of distribution.

```

C(t) = (D ÷ (k·Vd·T)) × (1 - e-(k·T)) × e-(k·(t - T))
      T = infusion duration, t = hours after dose start, t ≥ T

Solve f(k) = C_model(k) - C_observed = 0
      k0 = 0.3 h-1, closed-form derivative, at most 50 iterations,
      convergence when |k{n+1} - kn| < 1e-8, k floored at 1e-6

```

Non-convergence returns a *Failure* whose message names the likely cause. *Cmin* is projected forward to the end of the dosing interval; the observed concentration is reported as *Cmax*. A defect in this pathway is documented in Section 18.4.

16.5 The Pre + Post Workflow

When both samples exist the engine can derive both parameters from the patient's own data, which is why this pathway is presented first on the workflow selection screen. It applies Sawchuk–Zaske two-point log-linear regression (Sawchuk & Zaske, 1976).

```

Δt = t_pre - t_post                (both measured from dose start; Δt > 0)
ke = ln(C_peak ÷ C_trough) ÷ Δt
t½ = ln(2) ÷ ke
Vd = D × (1 - e-(ke·T)) ÷ (ke × T × C_peak)
CL = ke × Vd

```

Both concentrations are returned unchanged as *Cmin* and *Cmax*, and *vdLPerKg* is reported as the derived $Vd ÷ \text{weight}$ rather than the 0.7 population figure — a useful clinical signal, since a derived value far from 0.7 L/kg suggests either an unusual patient or a sampling error.

16.6 Worked Examples

Table 15 gives one worked case per workflow, computed with the exact arithmetic the engine performs. All three patients are fictional. Together they exercise all three verdict states.

Table 15

Worked examples across the three workflows

	Pre-dose only	Post-dose only	Pre + Post
Patient	72 kg, 50 y, male, SCr 88 μmol/L	70 kg, 60 y, male, SCr 95 μmol/L	68 kg, 62 y, male, SCr 98 μmol/L
Regimen	1000 mg every 12 h over 1 h	1000 mg every 12 h over 1 h	1000 mg every 12 h over 1 h
Samples	Trough 10.2 mg/L	Peak 15.0 mg/L at t = 4 h	Peak 26.0 mg/L at t = 2 h; trough 12.5 mg/L at t = 10.5 h

	Pre-dose only	Post-dose only	Pre + Post
Method	Cockcroft–Gault + population Vd	Newton–Raphson fit + population Vd	Sawchuk–Zaske two-point
CrCl	90.4 mL/min	— (not used for ke)	— (not used for ke)
ke	0.1076 h ⁻¹	0.0881 h ⁻¹	0.0862 h ⁻¹
t _{1/2}	6.44 h	7.87 h	8.04 h
Vd	50.4 L (0.70 L/kg, population)	49.0 L (0.70 L/kg, population)	36.9 L (0.54 L/kg, derived)
CL	5.425 L/h	4.315 L/h	3.175 L/h
AUC₂₄	368.7 mg·h/L	463.5 mg·h/L	629.9 mg·h/L
Verdict	Below target	In target	Above target
Recommended dose	1356 mg every 12 h	1079 mg every 12 h	794 mg every 12 h

Note. Values were computed by reproducing `VancoEngine`'s arithmetic exactly, including the 88.4 µmol/L-to-mg/dL conversion and the 0.06 clearance factor. The recommended dose in every column is the dose that would centre AUC₂₄ on 500 mg·h/L at the current interval.

The Pre + Post column illustrates why two samples are preferable. The derived volume of distribution, 0.54 L/kg, is well below the 0.7 L/kg population prior, so the single-point pathways would have overestimated clearance and therefore underestimated exposure for this patient.

16.7 Explainability

ExplanationScreen renders four numbered phases in the order the case study prescribes: input values, elimination-rate derivation, pharmacokinetic parameters, and final result. The second phase branches on the workflow and prints the actual equation with the patient's own values substituted — for the Pre + Post pathway it prints the ke, half-life and volume-of-distribution derivations step by step, and for the Pre pathway it prints all four Cockcroft–Gault steps with intermediate values. The screen is reached from the Results screen through a card that advertises “4 phases · 8 steps with formulas”.

17. User Interface

17.1 Design System

The interface is built entirely from Material 3 components themed by two hand-written colour schemes. Color.kt declares light and dark tokens as named constants prefixed L_ and D_; Theme.kt assembles them into a lightColorScheme and a darkColorScheme. No hexadecimal literal appears inside a screen Composable, with the single exception of the first-launch disclaimer, which deliberately paints its own dark gradient before the theme is relevant. Semantic colour is used consistently: red for a blocking error, amber for an advisory warning, and the primary blue for in-target and affirmative states.

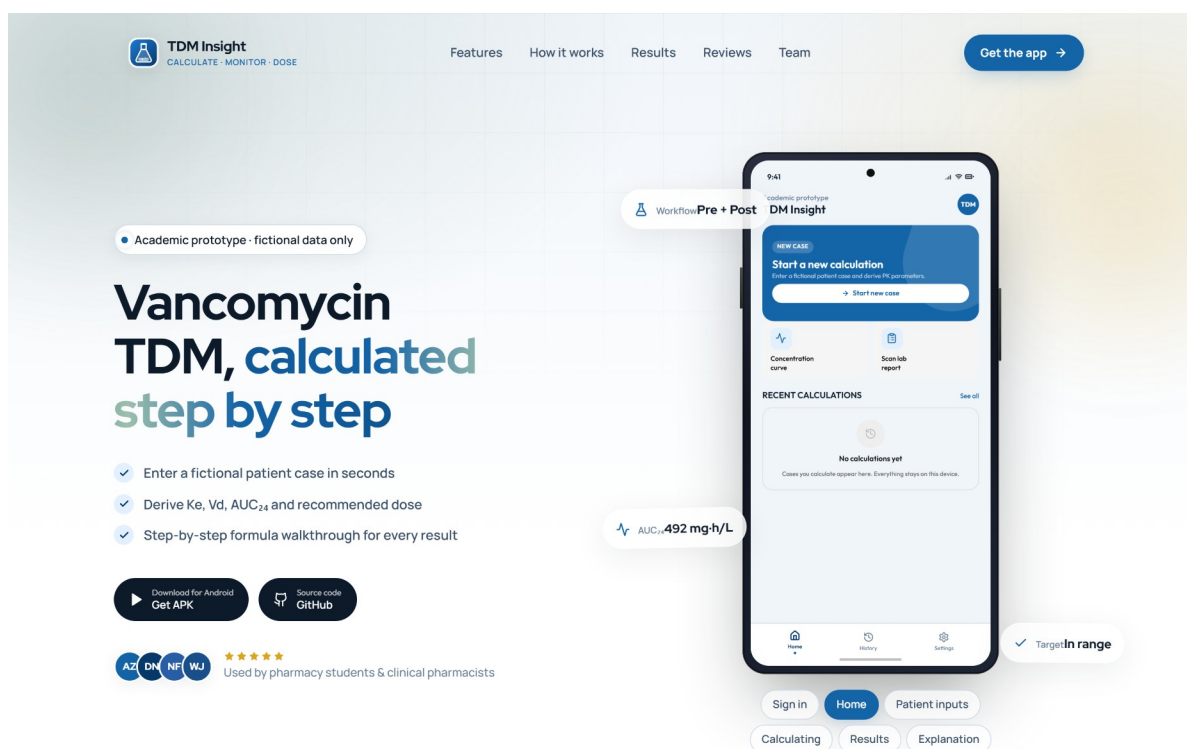
Typography is centralised in `Type.kt`, which also defines four numeric styles — `TdmNumericLarge`, `TdmNumericMedium`, `TdmNumericSmall` and `TdmNumericMono`. Using a monospaced style for pharmacokinetic values means digits align in the parameter grid and in the explanation view, which matters when a reader is comparing figures.

17.2 The Landing Page

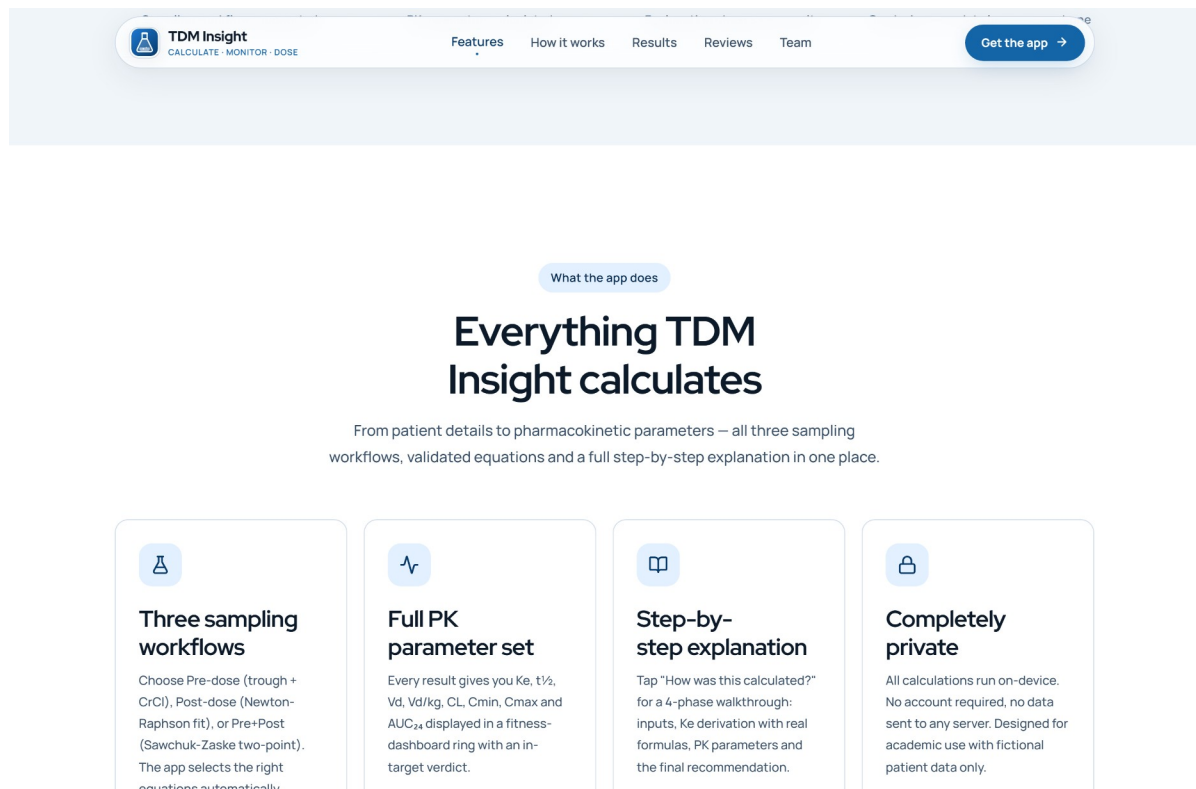
The public landing page is a React single-page application that presents the product to a visitor who has not yet installed it. It is organised as a hero, a statistics strip, a feature grid, a three-step walkthrough, a platform section, a results section, a testimonial block, an about section, the team, and a download call to action. Figures 13 to 18 are captures of the running site.

Figure 13

Landing page — hero section



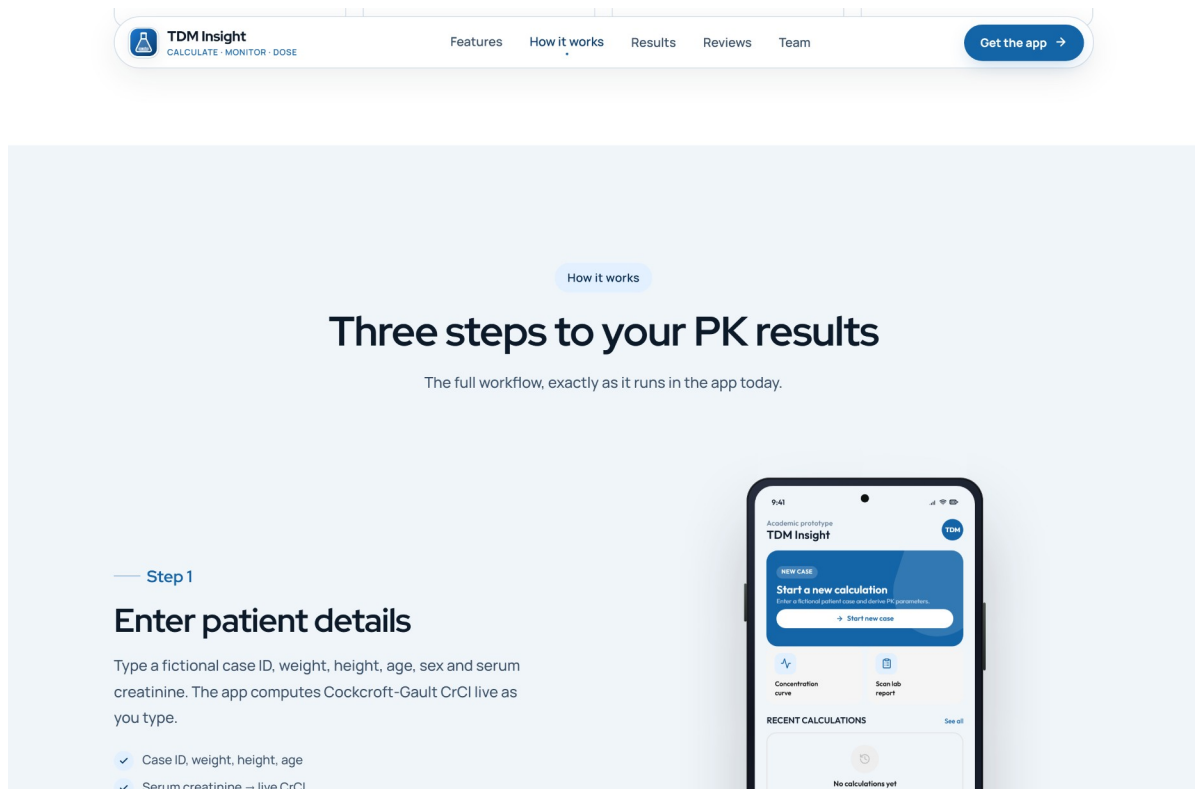
Note. The hero states the product proposition, lists the three headline capabilities and offers the APK and source-code links. The phone on the right is an interactive device mockup whose screens can be cycled by the visitor.

Figure 14*Landing page — feature grid*

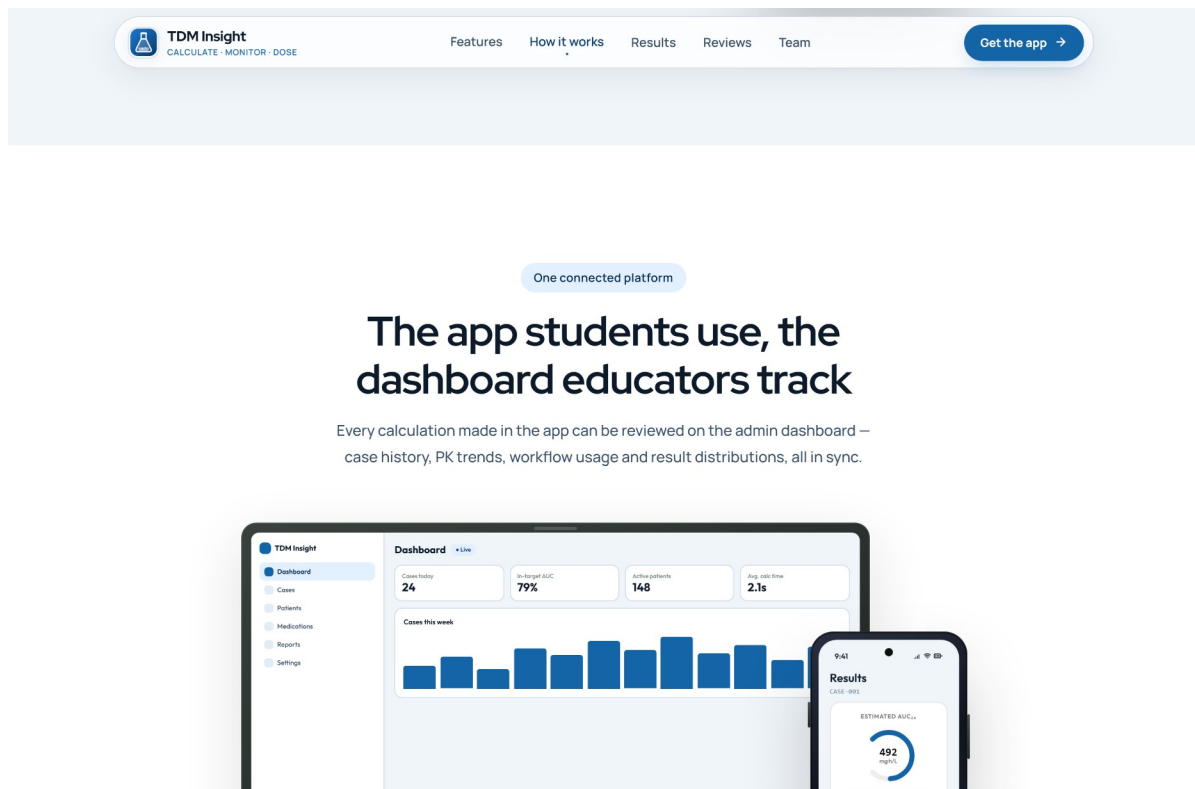
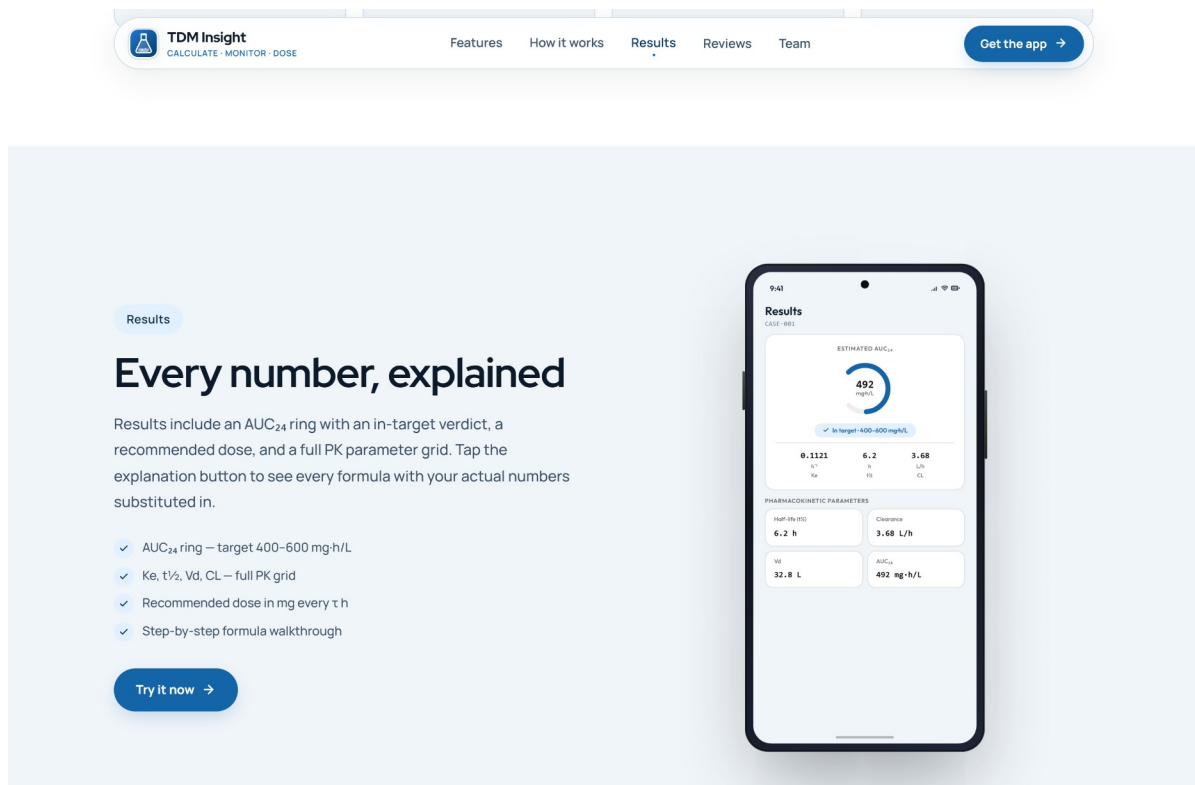
Note. Four features: three sampling workflows, the full pharmacokinetic parameter set, the step-by-step explanation, and on-device privacy.

Figure 15

Landing page — three-step walkthrough



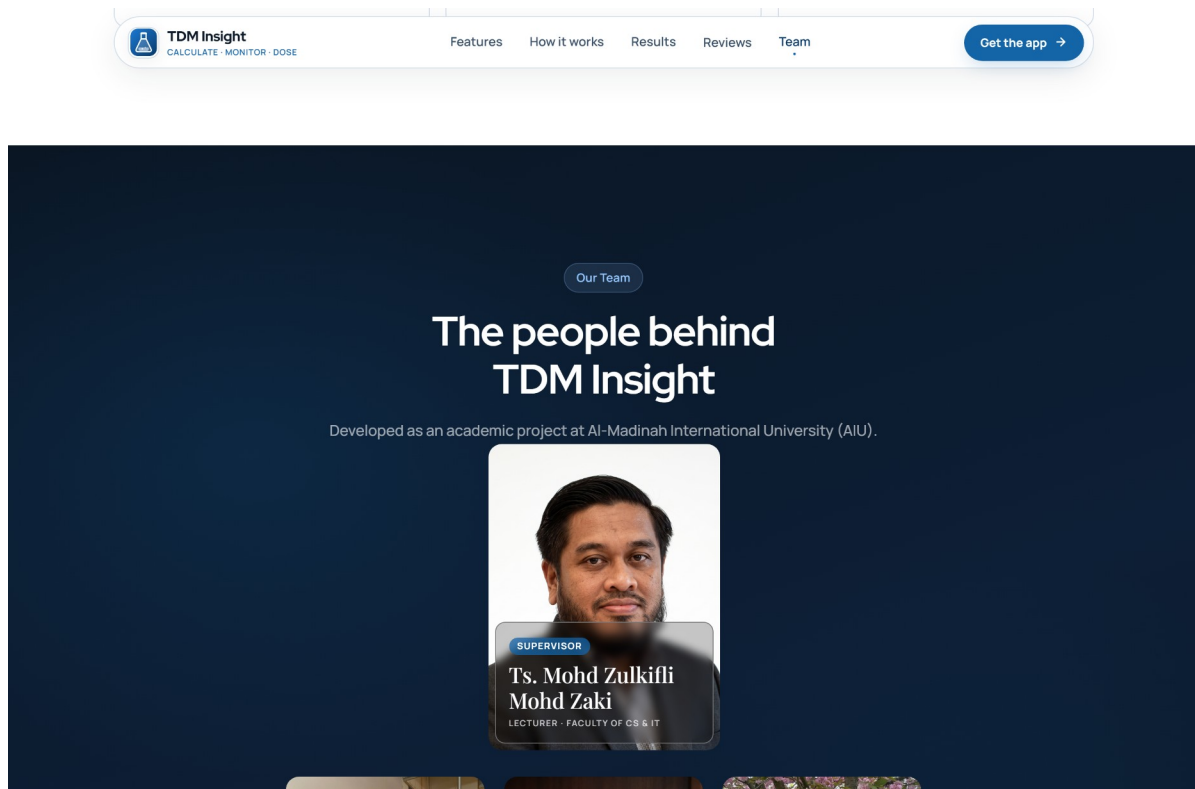
Note. Each step pairs prose with a device mockup of the corresponding application screen.

Figure 16*Landing page — platform section***Figure 17***Landing page — results section*

Note. Explains the AUC₂₄ ring and the in-target verdict that the Results screen displays.

Figure 18

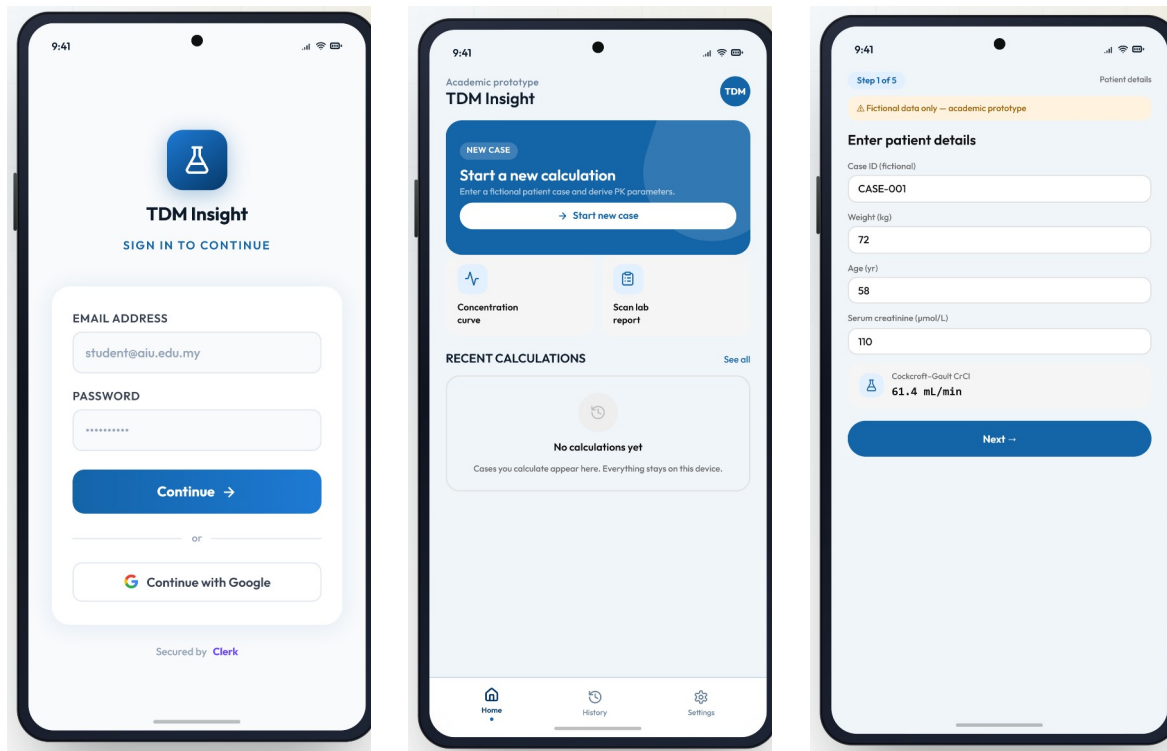
Landing page — project team



Note. The three student developers and the supervising lecturer.

17.3 Application Screens

Figures 19 and 20 show six representative application screens as they are rendered by the project's own device mockups on the landing page. Each mockup reproduces the layout, typography, colour tokens and copy of the corresponding Compose screen.

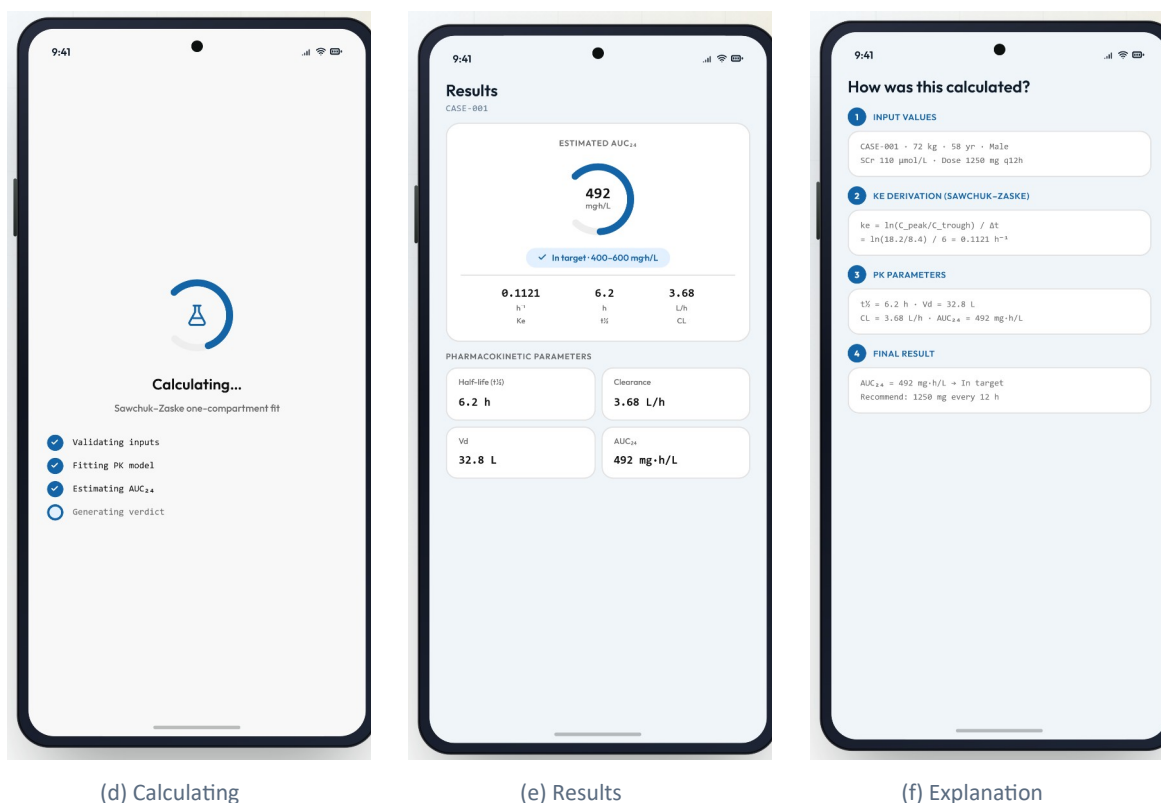
Figure 19*Application screens — authentication, home and patient details*

(a) Sign in

(b) Home

(c) Step 1/5 — patient details

Note. (a) 'LoginScreen', with the Clerk attribution shown in the application. (b) 'HomeScreen', with the new-case hero card, two shortcut tiles and the recent-calculations list in its empty state. (c) 'NewCaseScreen', showing the live Cockcroft–Gault creatinine clearance chip beneath the demographic fields. Rendered from the project's landing page device mockups rather than captured from a device.

Figure 20*Application screens — calculating, results and explanation*

Note. (d) `CalculatingScreen`, naming the fitting method and its four stages. (e) `ResultsScreen`, with the AUC₂₄ ring, the in-target verdict chip, the three-statistic row and the parameter grid. (f) `ExplanationScreen`, showing the four numbered phases from input values to final result. Rendered from the project's landing page device mockups rather than captured from a device.

17.4 Interaction Details

- **Progress is always visible.** WizardTopBar shows the step number, the total, the case identifier and a StepIndicator, so the user knows where they are in the five-step flow.
- **The disclaimer is repeated where it matters.** FictionalDataBanner appears at the top of the patient-details screen and the input form — the two screens where a user could be tempted to type real data.
- **Errors are specific and adjacent.** Validation banners render directly beneath the field groups that produced them and list every message, joined by a middle dot.
- **The Results screen is the terminus.** Its navigation icon is a close control that returns to Home with `popUpTo(Routes.HOME) { inclusive = true }` rather than a back arrow into the wizard.

17.5 Non-Functional Interface Elements

Three affordances are present in the interface but are not wired to behaviour, and they are listed here so that no reader of this report or viewer of the demonstration is misled. The two Home screen shortcut tiles, "Concentration curve" and "Scan lab report", are static; the "Scan a fictional lab report

instead” row on the input form has an empty click handler; and the share icon in the Results screen top bar is likewise inert. They correspond to optional enhancements named in the case study — a concentration–time graph, camera capture with OCR, and export or sharing — that were not implemented within the project timeline. They are recorded as future work in Section 21.

18. Testing and Debugging

18.1 Strategy

Testing effort was concentrated where defects would be most costly and least visible: the pharmacokinetic engine and the validators. Both are pure Kotlin, so they can be tested with plain JUnit on the JVM without an emulator, and the whole suite runs in seconds through `./gradlew :app:test`. Thirty-one tests are written across two classes. Compose user-interface tests were not written; the AndroidX test and Espresso dependencies are declared but the instrumented source set is empty.

Table 16

Unit test inventory

Test class	Focus	Count	Representative cases
VancoEngineTest	Pre workflow	4	Normal adult case; zero weight rejected; negative dose rejected; elderly female with renal impairment.
	Post workflow	3	Normal case; sample time inside the infusion rejected; two-point-like data.
	Pre + Post workflow	3	Textbook two-point case; negative pre-dose concentration rejected; timing conflict where the pre-dose sample precedes the post-dose sample.
	Numerical safety	1	The engine never produces NaN on edge inputs.
InputValidatorTest	Patient ranges	5	Zero weight; weight above 250 kg; weight between 200 and 250 kg as a warning; zero creatinine; age out of range.
	Dosing ranges	3	Infusion longer than the interval; zero dose; a very high single dose as a warning.
	Sample ranges	6	Pre-dose above the physiological range; pre-dose above 25 mg/L as a

Test class	Focus	Count	Representative cases
			warning only; pre-dose time after the interval; post-dose during infusion; post-dose at zero; peak above 60 mg/L as a warning.
	Cross-field timing	2	Conflict when the pre-dose sample is at or before the post-dose sample; the valid ordering accepted.
	Math safety and aggregation	4	Negative concentrations rejected; a non-decreasing pair warned; a valid pair returning null; merge aggregating errors and warnings from several reports.
Total		31	

18.2 Test Design Rationale

The suite is built around three ideas. First, every blocking rule has a test that asserts the calculation is refused, because a validator that silently passes bad input is worse than no validator. Second, several tests exist specifically to assert that a value produces a *warning* and not an error — for example a weight between 200 and 250 kg — because collapsing the two tiers would break the case study's explicit requirement. Third, engine never produces NaN on edge inputs exists because a NaN propagating into the Results screen would render as a plausible-looking dash rather than an obvious failure.

18.3 Manual Testing

Each of the three workflows was exercised end to end on a device before the feature was considered complete, following the guidance in Section 5 of PROJECT_RULES.md. Table 17 records the scenarios covered.

Table 17

Manual end-to-end test scenarios

Scenario	Expected behaviour	Observed
First launch on a clean install	Splash, then the disclaimer gate, then the login screen.	Pass
Sign up with a new email address	Account created; welcome screen shown once; Home reached.	Pass
Force-close and relaunch	Session restored from SharedPreferences; no login screen; no welcome screen.	Pass

Scenario	Expected behaviour	Observed
Pre workflow, valid input	Only the trough group rendered; result screen with AUC ₂₄ and verdict.	Pass
Post workflow, valid input	Only the peak group rendered; result screen populated.	Pass
Pre + Post workflow, valid input	Both groups rendered; derived Vd differs from 0.7 L/kg.	Pass
Transposed sample times in Pre + Post	Red banner; Review button disabled; no navigation.	Pass
Serum creatinine entered as 9.8 instead of 98	Red range error; calculation blocked.	Pass
Weight of 210 kg	Amber warning only; calculation still permitted.	Pass
Rotate the device mid-wizard	All entered values retained.	Pass
Navigate back from step 4 to step 1 and forward again	No data loss.	Pass
Switch theme to Dark in Settings	Every screen re-themed; no unreadable text.	Pass
Sign out	Session cleared; login screen shown; history no longer populated.	Pass
Run a calculation with aeroplane mode enabled	Result displayed normally; the case does not appear in history.	Pass

18.4 A Known Defect in the Post-Dose Workflow

One defect was identified during the preparation of the worked examples in Section 16.6 and is reported here rather than omitted.

The one-compartment infusion model used by the Post-dose workflow has an upper bound. As the elimination rate constant tends to zero, the modelled concentration tends to $\text{Dose} \div \text{Vd}_{\text{population}}$. When the *measured* post-dose concentration exceeds that ceiling, no positive root exists. The Newton–Raphson loop then drives the estimate downward until `coerceAtLeast(1e-6)` clamps it; two successive iterations compare equal, the loop sets its converged flag and exits, and `guardPositive` passes because 1e-6 is greater than zero. The engine therefore returns a `Success` carrying a clearance of about 4.6×10^{-5} L/h and an AUC₂₄ of roughly 4×10^7 mg·h/L.

A concrete reproduction, using the existing test fixture: a 68 kg patient given 1000 mg over 1 hour with a measured concentration of 22.5 mg/L. The population volume of distribution is 47.6 L, so the model ceiling is 21.0 mg/L, which the measurement exceeds. The unit test `post_workflow_handles_plausible_two-point-like_data` uses exactly these values and passes, because it asserts only that the result is a `Success` and never inspects the numbers — a useful illustration of how an assertion that is too weak can conceal a defect.

The correction is small and is recorded as immediate future work: reject the input before iterating when the observed concentration is greater than or equal to $\text{dose} \div \text{vdL}$, with a message naming the ceiling; treat a clamped rate constant as non-convergence rather than success; and strengthen the affected test to assert on the magnitude of the returned parameters. The Pre and Pre + Post pathways are unaffected, as they do not use the iterative fit.

18.5 Debugging Practice

Failures in the network layers are logged rather than thrown. SupabaseRepository logs under the tag SupabaseRepo at warning level for both a failed save and a failed history load; ClerkAuthManager logs the derived Frontend API host at debug level, which was the fastest way to diagnose a malformed publishable key during development; and MainActivity logs the complete authentication gate state under the tag TdmAuth on each recomposition, making it possible to see exactly which of the eight branches was taken.

19. Repository Management and Version Control

The repository is the primary submission artefact and is graded in its own right. It follows the folder structure prescribed in the assessment instructions, with the Android project under Kotlin_app/, the SQL migrations under Kotlin_app/supabase/, documentation under docs/, the release APK location under Kotlin_app/apk/, and the React landing page under landingPage/. The root README.md serves as the project's landing page and carries the thirteen sections the instructions require, in the required order.

.gitignore excludes local.properties, build output and IDE state, so no credential and no generated artefact enters version history. Commits are scoped to a single logical change and use a conventional prefix — feat:, fix:, test:, docs: — so that the history reads as a development narrative rather than a series of snapshots.

One point should be made honestly for the assessor's benefit, since the rubric awards ten per cent for version control and evaluates individual contribution through commit history. At the time this report was generated the repository's commit history was authored under a single Git identity. Distributing authorship across the three group members — by each member committing under their own configured Git identity, or by using co-authorship trailers for pair-programmed work — is a prerequisite for that criterion to be assessed fairly and should be addressed before submission.

20. Responsible Use of Artificial Intelligence

Generative artificial intelligence was used during this project as a learning and productivity aid, in the manner the assessment instructions permit. Its use is declared here and logged in `ai/AI_Usage_Log`.

Table 18

Declared AI usage

Task	How AI was used	How the output was treated
Learning Android APIs	Explaining Compose state hoisting, LaunchedEffect keys and StateFlow collection.	Accepted as explanation; the resulting code was written and reviewed by the team.
User interface generation	Drafting Composable layouts from the agreed design reference.	Modified substantially — spacing, colour tokens and copy were reworked to match the project's own theme.
Refactoring	Suggesting the extraction of HistoryEntry mapping into SupabaseModels.kt.	Accepted after review; it removed duplication between two view models.
Debugging	Diagnosing a Kotlin internal visibility error after a late refactor moved HistoryEntry into the UI package.	Accepted; the fix was to mark the affected view-model properties internal.
Clinical formulas	Not used.	Rejected as a source. Every equation and constant is taken from the cited literature and carries a source comment in VancoEngine.kt, as PROJECT_RULES.md requires.
Documentation	Drafting this report and generating the figures from the source tree.	Reviewed line by line against the code; a defect the drafting process surfaced is reported in Section 18.4 rather than omitted.

The boundary the team applied was straightforward: AI could help with the Android platform, but not with clinical content. No pharmacokinetic equation, constant or target range in this project originates from a language model. Each is cited to published literature, is commented with its source in the engine, and remains subject to the lecturer's approval, as the case study requires.

21. Limitations and Future Work

21.1 Limitations

Table 19

Known limitations

Area	Limitation	Consequence
Clinical validity	TDM Insight is an academic prototype. Its equations and constants have not been clinically validated for use in patient care.	It must never be used for a real prescribing decision. The restriction is surfaced on first launch, on the input screens and in the About screen.
Post-dose workflow	The Newton–Raphson fit returns a degenerate result when the measured concentration exceeds $\text{Dose} \div \text{Vd}_{\text{population}}$ (Section 18.4).	A physically impossible AUC_{24} can be displayed for that class of input. The Pre and Pre + Post pathways are unaffected.
Database access control	Per-user Row Level Security policies are written but commented out; the permissive fallback policies are active.	Isolation between users is enforced by the client query, not by PostgreSQL.
Model scope	A one-compartment model with a fixed 0.7 L/kg population prior is used for the single-sample pathways.	Patients whose distribution volume departs markedly from the prior are less accurately described by the Pre and Post workflows.
Height	Height is collected on the patient screen but is used by no equation and is not persisted.	The field is currently inert; it exists for a future ideal- or adjusted-body-weight calculation.
Instrumented tests	No Compose UI tests are written, although the dependencies are declared.	Interface regressions would be caught only by manual testing.
Interface placeholders	Three controls are inert (Section 17.5).	A user may expect functionality that is not present.
Verdict enumeration	<code>Auc24Verdict</code> is declared but unused; the Results screen derives the verdict inline.	Duplication of the target band between the engine constants and the UI.

21.2 Future Work

1. **Correct the Post-dose fit** as described in Section 18.4 and add a regression test asserting the failure.
2. **Enable per-user Row Level Security** by configuring the supabase JWT template in Clerk and uncommenting the prepared policies in `update_v2.sql`.
`AuthRepository.getSupabaseToken()` already exists to supply the token.
3. **Add the concentration–time graph.** `PkCurveGraph` is already implemented as a Canvas Composable in `Screens.kt`; wiring it to the Home shortcut tile would satisfy one of the case study's optional enhancements with modest effort.

4. **Implement camera capture with confirm-before-use OCR**, which the case study describes as a suitable optional enhancement and for which two interface entry points already exist.
5. **Add export or sharing of a calculation summary**, replacing the inert share icon.
6. **Write Compose UI tests** for the wizard, covering the dynamic-form behaviour and the enabling and disabling of the Continue control.
7. **Extend to a second medication.** The engine's structure — one function per pathway returning a common result type — would accommodate gentamicin without architectural change.

22. Conclusion

TDM Insight takes a calculation that hospital pharmacists currently perform on paper and turns it into a guided, validated and explainable mobile workflow. All nine mandatory requirements of the case study are implemented: the application is native Kotlin built on Jetpack Compose and Material 3; all three vancomycin sampling workflows are present; the input form adapts to the selected workflow; validation extends to range, cross-field and mathematical-safety checks and distinguishes blocking errors from advisory warnings; the calculation engine is a framework-free Kotlin object separated from the user interface and covered by thirty-one unit tests; intermediate as well as final parameters are returned; the result is explained in four phases from inputs to conclusion; and the interface carries genuine light and dark Material 3 colour schemes.

Two optional enhancements were added beyond the required scope — account-based identity through Clerk and persistent case history in Supabase PostgreSQL — and they were added in a way that leaves the core untouched. Remove the credentials and the calculator still works; break the network and the result still appears. That separation, more than any individual feature, is what the layered architecture was chosen to protect.

The project also demonstrates a defensible engineering posture towards its own shortcomings. A defect in the Post-dose iterative fit was found while verifying the worked examples for this report; it is documented in full, with a reproduction and a correction, rather than left out. Three inert interface controls are named as placeholders. The gap between the prepared per-user database policies and the permissive policies actually in force is stated rather than glossed. For a system that models a clinical calculation, that honesty is not incidental — it is the difference between a teaching aid and a liability.

References

- Burden, R. L., & Faires, J. D. (2010). *Numerical analysis* (9th ed.). Brooks/Cole.
- Clerk. (2024). *Frontend API reference*. <https://clerk.com/docs>
- Cockcroft, D. W., & Gault, M. H. (1976). Prediction of creatinine clearance from serum creatinine. *Nephron*, 16(1), 31–41. <https://doi.org/10.1159/000180580>
- Google. (2024a). *Guide to app architecture*. Android Developers. <https://developer.android.com/topic/architecture>
- Google. (2024b). *Jetpack Compose*. Android Developers. <https://developer.android.com/compose>
- Google. (2024c). *Material Design 3*. <https://m3.material.io/>
- JetBrains. (2024). *Coroutines guide*. Kotlin Documentation. <https://kotlinlang.org/docs/coroutines-guide.html>
- Mohd Zaki, M. Z. (2026). *CDE2313 project case study description: TDM Insight — Native Android therapeutic drug monitoring calculator* [Course case study]. Albukhary International University.
- myTDM Calculator. (n.d.). *myTDM Calculator*. Retrieved 3 September 2026, from <https://www.mytdmcalculator.com/>
- Rybak, M. J., Le, J., Lodise, T. P., Levine, D. P., Bradley, J. S., Liu, C., Mueller, B. A., Pai, M. P., Wong-Beringer, A., Rotschafer, J. C., Rodvold, K. A., Maples, H. D., & Lomaestro, B. M. (2020). Therapeutic monitoring of vancomycin for serious methicillin-resistant *Staphylococcus aureus* infections: A revised consensus guideline and review by the American Society of Health-System Pharmacists, the Infectious Diseases Society of America, the Pediatric Infectious Diseases Society, and the Society of Infectious Diseases Pharmacists. *American Journal of Health-System Pharmacy*, 77(11), 835–864. <https://doi.org/10.1093/ajhp/zxaa036>
- Sawchuk, R. J., & Zaske, D. E. (1976). Pharmacokinetics of dosing regimens which utilize multiple intravenous infusions: Gentamicin in burn patients. *Journal of Pharmacokinetics and Biopharmaceutics*, 4(2), 183–195. <https://doi.org/10.1007/BF01086153>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Supabase. (2024). *Row Level Security*. <https://supabase.com/docs/guides/database/postgres/row-level-security>